源程序文档

软件名称：祁连山电竞陪玩服务平台软件 V1.0

项目	内容
文档用途	计算机软件著作权登记 / 电子版权认证 送审材料之一（源程序文档）
送审口径	前 30 页 + 后 30 页，每页 50 行，共 60 页 / 3000 行
本文件实际页数	60 页（前 30 页 + 后 30 页）
本文件实际展示行数	3000 行（非空行口径，逐页恰好 50 行）
语料来源	祁连山电竞陪玩服务平台软件 服务端源码（TypeScript / NestJS），取自仓库 `server/src/**`
抽取方式	按业务核心模块选取真实源文件与真实行区间，未改写任何业务语句；仅对注释中出现的本地环境文件名做等价替换
页首标注	每页首行固定为「软件名称：祁连山电竞陪玩服务平台软件 V1.0」
页尾标注	每页末尾固定为「--- 第 N 页 ---」

说明：本段为封面说明，不计入 60 页正文。正文自「第一部分 前 30 页」起算。

取用说明（前 30 页取什么、后 30 页取什么）

按照软件著作权登记「源程序前 30 页 + 后 30 页」的通行要求，本文档把全部送审源码视为一个按模块顺序拼接的连续源程序清单：

- **前 30 页（第 1-30 页）= 程序头部**：服务入口与根模块装配 → 统一响应 / 异常处理 / 认证守卫等基础设施 → 认证与授权模块（注册、登录、会话轮换、登出、当前主体、三类会话令牌签发）→ 伴随会话（打手态）每请求校验。
- **后 30 页（第 31-60 页）= 程序尾部**：积分账务 → 订单状态机与完成结算 / 退款 → 会话与客服大厅 → 消息推送 → 充值人工审核 → 平台抽成与金额分账 → 数据实体定义。
- **分页口径**：每页固定 50 个非空行；跨页处保留源文件路径与行区间标注，审查人员可按标注回溯原始文件。
- **省略部分**：第 30 页与第 31 页之间省略了本软件源程序中段，省略说明见正文中部。

口径澄清（避免与申请表中的「源程序量」混淆）：本文档按登记要求只提交 **3000 行**（60 页 × 50 行）作为送审样本；**本软件的实际源程序量远大于此**——服务端 `server/src/**` 为 280 个 TypeScript 文件 / 27,100 行（非空行口径），三端（服务端 + 管理端 + 老板端客户端与共享包）合计 **71,807 行**（物理行 78,398 行）。申请表中的「源程序量」应填写实际总量，统计命令与原始输出见《软著申请表信息.md》。

抽取清单（真实文件与真实行区间）

序号	位置	源文件	采用行区间	本文件展示行数	片段说明
1	前 30 页	`server/src/main.ts`	1-148	135	服务入口：全局前缀 /api/v1、统一

					校验管道、安全响应头、Swagger、媒体路由排除
2	前 30 页	server/src/app.module.ts	1-105	103	根模块：TypeORM 数据源、Redis、限流、全部业务模块装配
3	前 30 页	server/src/common/interceptors/response.interceptor.ts	1-30	27	统一响应包装（成功码 + 数据体）
4	前 30 页	server/src/common/filters/all-exceptions.filter.ts	1-89	82	全局异常过滤器：不外泄堆栈、中文业务错误、requestId 回传
5	前 30 页	server/src/common/guards/jwt-auth.guard.ts	1-288	274	JWT 守卫：单设备会话、受限会话（打手态 / 充值管理员）允许清单、账号封禁实时校验
6	前 30 页	server/src/modules/auth/auth.controller.ts	1-190	174	认证接口：注册、登录、刷新、登出、当前主体
7	前 30 页	server/src/modules/auth/auth.service.ts	26-172	140	认证服务：令牌 TTL 常量、会话种类归一化、注册
8	前 30 页	server/src/modules/auth/auth.service.ts	250-396	133	认证服务：邮箱验证码重置、登录（统一失败文案）、封禁校验
9	前 30 页	server/src/modules/auth/auth.service.ts	402-531	128	认证服务：身份事件留痕、refresh 轮换
10	前 30 页	server/src/modules/auth/auth.service.ts	595-667	71	认证服务：登出与会话种类解析
11	前 30 页	server/src/modules/auth/auth.service.ts	680-760	80	认证服务：当前主体（按角色附带档案）
12	前 30 页	server/src/modules/auth/auth.service.ts	762-895	127	认证服务：令牌签发（primary / companion / recharge 三态）
13	前 30 页	server/src/modules/auth/companion-session.check.t	1-27（节选）	26	伴随会话每请求校验：绑定有效、打

		s			手在职、双方未封禁
14	后 30 页	server/src/modules/wallet/wallet.service.ts	1-291	277	积分钱包：管理员发放 / 回收、幂等键、余额快照、工作端收益
15	后 30 页	server/src/modules/order/order.service.ts	94-168	70	订单状态机：合法迁移表（唯一依据）
16	后 30 页	server/src/modules/order/order.service.ts	468-570	95	订单完成结算事务：行锁 + 三重幂等 + 服务端分账
17	后 30 页	server/src/modules/order/order.service.ts	1167-1265	97	退款事务：冲正结算 + 回退打手战绩（同一事务）
18	后 30 页	server/src/modules/chat/chat.service.ts	416-541	125	会话列表：三类分区与未读汇总
19	后 30 页	server/src/modules/chat/chat.service.ts	706-920	202	客服大厅：待接入池、抢占接入、释放回池
20	后 30 页	server/src/modules/chat/chat.service.ts	1015-1146	123	消息发送：成员校验、限流、序号分配、实时推送
21	后 30 页	server/src/modules/recharge/recharge.service.ts	369-920	511	充值人工审核：待审列表、发放（复用积分发放事务）、驳回、收款码管理

未纳入本文档的模块（中部省略内容，见「省略说明」）

模块	所在路径	说明
数据库结构与迁移脚本	server/src/migrations/**	55 个迁移文件，逐版本建表 / 改列 / 加索引
管理端接口与 RBAC	server/src/modules/admin/**、rbac/**、log/**	管理员角色、权限点、角色授权、操作日志与登录日志
商品与目录	server/src/modules/game/**、product/**、division/**、club/**	游戏 / 分部 / 分类 / 商品及其多图视频
媒体资源	server/src/modules/media/**	魔数 + 扩展名 + 内容解析三重校验、哈希去重、公开与受保护双通道
排行榜与统计	server/src/modules/leaderboard/**	分部榜、周榜月榜、结算流水聚合重算
系统设置	server/src/modules/setting/**	高安全配置仅超级管理员可写、改完实

		时生效
站内通知与公告	`server/src/modules/home/**`、`tip/**`、`feedback/**`、`complaint/**`	通知下发、公告、投诉与反馈
定时任务	`server/src/schedule/**`	派单超时回退、待接单超时、榜单汇总
智能问答	`server/src/modules/ai/**`	会话按次轮换、系统提示词按结算口径动态填充
客户端与管理端	`apps/boss-app/lib/**`、`apps/packages/core/lib/**`、`admin-web/src/**`	Flutter 客户端与 Vue3 管理端；如需补充可按同一抽取口径补交

— — —

第一部分 前 30 页（程序头部）

软件名称：祁连山电竞陪玩服务平台软件　V1.0
`===== 源文件：server/src/main.ts　（第 1-148 行）=====`
`----- 片段说明：服务入口：全局前缀 /api/v1、统一校验管道、安全响应头、Swagger、媒体路由排除 -----`

```
import { Logger, RequestMethod } from '@nestjs/common';
import { NestFactory } from '@nestjs/core';
import { DocumentBuilder, SwaggerModule } from '@nestjs/swagger';
import { IoAdapter } from '@nestjs/platform-socket.io';
import compression from 'compression';
import * as express from 'express';
import { existsSync } from 'fs';
import { join } from 'path';
import { AppModule } from './app.module';
import { AllExceptionsFilter } from './common/filters/all-exceptions.filter';
import { PublicCacheInterceptor } from './common/interceptors/public-cache.interceptor';
import { ResponseInterceptor } from './common/interceptors/response.interceptor';
import { LenientValidationPipe } from './common/pipes/lenient-validation.pipe';
async function bootstrap() {
// 生产安全闸（负责人 2026-10-05 邮箱验证）：console 发信驱动不发真信，只允许开发/自动化测试使用。
// 若生产环境误配 MAIL_DRIVER=console，用户会看到「验证码已发送」但永远收不到邮件 —— 这是最隐蔽的故障，
// 因此在启动阶段直接拒绝启动（fail fast），而不是留到线上靠用户投诉发现。
//
// 注意：pm2 的 ecosystem.config.js 会注入 NODE_ENV=production（本项目"生产"= 本机 pm2 常驻）。
// 因此自动化测试需要 console 驱动时，必须显式加 MAIL_ALLOW_CONSOLE=true 这道第二把钥匙：
// 单靠 MAIL_DRIVER=console 起不来，避免"误配一行就静默不发信"。
if (
process.env.NODE_ENV === 'production' &&
String(process.env.MAIL_DRIVER ?? 'smtp').toLowerCase() === 'console' &&
String(process.env.MAIL_ALLOW_CONSOLE ?? 'false').toLowerCase() !== 'true'
) {
throw new Error(
'MAIL_DRIVER=console 禁止在生产环境使用（它不会真的发信，请改为 smtp）；' +
'确需在测试中启用请显式设置 MAIL_ALLOW_CONSOLE=true',
);
}
const app = await NestFactory.create(AppModule);
// Socket.IO 适配器（docs/07：/ws/chat 实时网关，派单通知与聊天共用）
app.useWebSocketAdapter(new IoAdapter(app));
// 响应压缩（负责人 2026-10-04：继续更快）——JSON/文本走 gzip，隧道带宽占用降到约 1/5
//
// threshold 从 1024 下调到 512（2026-10-04 性能专项实测）：
// 8 个关键公开接口里只有 /public/products（2.5KB）超过 1KB，其余（/divisions 1276B、
// /public/home 877B、/public/players 403B…）全部不低于阈值也全部没压缩。
// 而"是否压缩"由客户端是否发 Accept-Encoding 决定（Express compression 只看请求头），
// 客户端侧已统一补上该请求头，这里把门槛降低让中小响应也吃上压缩。
// 512 是权衡值：更小的响应（<512B，如 /public/leaderboard 245B）压缩后反而更大
//　（gzip 头约 20 字节 + CPU），不值得。
app.use(compression({ threshold: 512 }));
// 全局前缀 /api/v1；公开媒体排除在外：客户端统一使用 /media/public/:id 直链
```

```
// （修复：此前全部媒体 URL 生成时不含前缀，真机/隧道访问 404 导致头像/背景/商品图/论坛媒体全部加载失败）
app.setGlobalPrefix('api/v1', {
exclude: [{ path: 'media/public/(.*)', method: RequestMethod.GET }],
});
app.enableCors({
--- 第 1 页 ---
```

软件名称：祁连山电竞陪玩服务平台软件 **V1.0**

```
origin: process.env.CORS_ORIGINS ? process.env.CORS_ORIGINS.split(',') : true,
credentials: true,
});
// 全局管道：白名单校验，拒绝未知字段（docs/03 安全铁律）
// 更新类/简介类 DTO 继承 LenientUpdateDto 后，本管道会对其放宽为「未知字段静静忽略」
//  （见 common/pipes/lenient-validation.pipe.ts；其余接口行为完全不变）
app.useGlobalPipes(new LenientValidationPipe());
// 公开字典类接口服务端微缓存（30s）——挡掉大部分重复查询
app.useGlobalInterceptors(new PublicCacheInterceptor());
app.useGlobalInterceptors(new ResponseInterceptor());
app.useGlobalFilters(new AllExceptionsFilter());
// 托管管理端（admin-web/dist）：一个地址同时提供管理端页面 + API + 媒体，
// 外网（Cloudflare 隧道/局域网）访问根路径即为管理后台，无需单独部署前端（docs/11）
const adminDist = join(__dirname, '..', '..', 'admin-web', 'dist');
// 请求日志（2026-09-25 排查消息页空白问题加入，注释写"定位后移除"但一直留着）
//
// 2026-10-04 性能专项：生产环境（NODE_ENV=production，pm2 当前就是这个）每个 API 请求
// 都同步写一行日志 —— 高频轮询（聊天 sync、订单列表 SWR 回源）下这是实打实的额外开销，
// 且线上日志没有排查价值（本地开发才有）。因此仅在非生产环境保留。
if (process.env.NODE_ENV !== 'production') {
app.use((req: express.Request, res: express.Response, next: express.NextFunction) => {
const t0 = Date.now();
res.on('finish', () => {
if (req.path.startsWith('/api/')) {
Logger.log(`${req.method} ${req.path} → ${res.statusCode} (${Date.now() - t0}ms)`, 'HTTP');
}
});
next();
});
}
// 官网静态页（负责人 2026-09-25：俱乐部官网，内容由 /api/v1/public/site 驱动）
const siteDist = join(__dirname, '..', '..', 'site');
if (existsSync(join(siteDist, 'index.html'))) {
app.use('/site', express.static(siteDist));
}
// UI 设计稿评审页（2026-10-03：改版设计图供负责人在线审批，纯静态，不进数据库）
const designDist = join(__dirname, '..', '..', 'site', 'design');
if (existsSync(join(designDist, 'index.html'))) {
app.use('/design', express.static(designDist));
// 已撤下的设计图必须返回 404，不能被下面的 SPA 回退成管理端 HTML
//  （否则浏览器里显示 200 + HTML，表现为破图/空白，极难排查）
app.use('/design', (req: express.Request, res: express.Response) => {
res.status(404).json({ code: 404, message: 'Not Found', data: null });
});
}
if (existsSync(adminDist)) {
app.use(express.static(adminDist, { index: false }));
app.use((req: express.Request, res: express.Response, next: express.NextFunction) => {
if (req.method !== 'GET' && req.method !== 'HEAD') return next();
const p = req.path;
```

```
// API / 媒体 / WebSocket / Swagger 交给 Nest 处理，不做 SPA 回退
if (p.startsWith('/api/') || p.startsWith('/media/') || p.startsWith('/socket.io') || p.startsWith('/api-docs')) {
return next();
}
// 敏感路径防护（docs/09）：点文件段（/环境变量配置、/.git…）与常见配置文件名不做 SPA 回退，一律 404
const sensitiveFiles = new Set([
'.env', 'package.json', 'package-lock.json', 'yarn.lock', 'composer.json',
'dockerfile', 'docker-compose.yml', 'docker-compose.yaml', '.gitignore', '.npmrc', 'server.key', 'server.crt',
]);
const segs = p.split('/').filter(Boolean).map((s) => s.toLowerCase());
if (segs.some((s) => s.startsWith('.')) || segs.some((s) => sensitiveFiles.has(s))) {
res.status(404).json({ code: 404, message: 'Not Found', data: null });
return;
}
res.sendFile(join(adminDist, 'index.html'));
});
}
// Swagger 仅非生产环境开启（docs/03）
if (process.env.NODE_ENV !== 'production') {
const config = new DocumentBuilder()
.setTitle('平台 API')
.setDescription('老板端 / 工作端 / 管理后台统一 API（/api/v1）。规范见 docs/03。')
.setVersion('0.1.0')
.addBearerAuth()
.build();
const document = SwaggerModule.createDocument(app, config);
SwaggerModule.setup('api-docs', app, document);
}
// 3100 为 FISH 专属端口（3000 被本机 XinQiao nssm 服务占用，docs/11 铁律）
const port = Number(process.env.PORT ?? 3100);
// 监听 0.0.0.0：手机真机与 Cloudflare Tunnel 均依赖
await app.listen(port, '0.0.0.0');
console.log(`[server] listening on http://0.0.0.0:${port} (NODE_ENV=${process.env.NODE_ENV ?? 'development'})`);
}
void bootstrap();
===== 源文件：server/src/app.module.ts （第 1-105 行）=====
----- 片段说明：根模块：TypeORM 数据源、Redis、限流、全部业务模块装配 -----
import { MiddlewareConsumer, Module, NestModule } from '@nestjs/common';
import { ConfigModule, ConfigService } from '@nestjs/config';
import { JwtModule } from '@nestjs/jwt';
import { TypeOrmModule } from '@nestjs/typeorm';
import { MailModule } from './common/mail/mail.module';
import { GlobalRateLimitMiddleware } from './common/middleware/global-rate-limit.middleware';
import { RequestIdMiddleware } from './common/middleware/request-id.middleware';
import { validateEnv } from './config/env.validation';
import { AdminModule } from './modules/admin/admin.module';
import { AiModule } from './modules/ai/ai.module';
import { AppUpdateModule } from './modules/app-update/app-update.module';
import { AuthModule } from './modules/auth/auth.module';
import { GameModule } from './modules/game/game.module';
import { ClubModule } from './modules/club/club.module';
import { DivisionModule } from './modules/division/division.module';
--- 第 3 页 ---
```

```
import { PlayersModule } from './modules/worker/players.module';
import { WorkerPermissionModule } from './modules/worker/worker-permission.module';
import { HealthModule } from './modules/health/health.module';
import { HomeContentModule } from './modules/home/home.module';
import { IdentityBindingModule } from './modules/identity-binding/identity-binding.module';
import { ChatModule } from './modules/chat/chat.module';
import { ComplaintModule } from './modules/complaint/complaint.module';
import { FeedbackModule } from './modules/feedback/feedback.module';
import { FriendModule } from './modules/friend/friend.module';
import { LeaderboardModule } from './modules/leaderboard/leaderboard.module';
import { LevelModule } from './modules/level/level.module';
import { LogModule } from './modules/log/log.module';
import { MediaModule } from './modules/media/media.module';
import { SystemSettingModule } from './modules/setting/system-setting.module';
import { RealtimeModule } from './realtime/realtime.module';
import { OrderModule } from './modules/order/order.module';
import { ProductModule } from './modules/product/product.module';
import { RechargeModule } from './modules/recharge/recharge.module';
import { RefundModule } from './modules/refund/refund.module';
import { TipModule } from './modules/tip/tip.module';
import { UserModule } from './modules/user/user.module';
import { WalletModule } from './modules/wallet/wallet.module';
import { SiteModule } from './modules/site/site.module';
@Module({
imports: [
ConfigModule.forRoot({ isGlobal: true, validate: validateEnv }),
// 全局 JWT（守卫跨模块使用；签发时按 access/refresh 显式指定密钥，docs/04）
JwtModule.register({ global: true }),
TypeOrmModule.forRootAsync({
inject: [ConfigService],
useFactory: (config: ConfigService) => ({
type: 'mysql',
host: config.get<string>('DB_HOST'),
port: config.get<number>('DB_PORT'),
username: config.get<string>('DB_USER'),
password: config.get<string>('DB_PASSWORD'),
database: config.get<string>('DB_NAME'),
autoLoadEntities: true, // 各模块 forFeature 自动注册实体
charset: 'utf8mb4',
timezone: 'Z',
synchronize: false, // 铁律：一律走 migration（docs/10）
}),
}),
HealthModule,
AuthModule,
// 邮件通道（负责人 2026-10-05：邮箱验证替代短信验证；全局提供 MailerService）
MailModule,
AdminModule,
// AI 咨询「祁小山」（负责人 2026-10-04）：老板端底部中间按钮入口
AiModule,
```

AppUpdateModule,
WalletModule,
GameModule,
DivisionModule,
ClubModule,
PlayersModule,
WorkerPermissionModule,
ProductModule,
OrderModule,
HomeContentModule,
MediaModule,
SystemSettingModule,
SiteModule,
RealtimeModule,
ChatModule,
FriendModule,
FeedbackModule,
// 顶部导航三入口（负责人 2026-10-04）：打赏（积分）/ 退款申请 / 投诉工单
TipModule,
RefundModule,
ComplaintModule,
UserModule,
// 身份绑定（老板 ↔ 打手，负责人 2026-10-06 方案甲）：管理端**只读**查询接口（运营排查）
IdentityBindingModule,
// 充值（线下收款 + 人工审核，负责人 2026-10-06）：老板端提交申请 + 受限管理态审核 + 管理端配置。
// **合规红线**：不接任何支付通道、不做自动到账、不做提现、平台不经手资金 —— 见 CLAUDE.md 结构规则 12。
RechargeModule,
LeaderboardModule,
LevelModule,
LogModule,
],
})
export class AppModule implements NestModule {
configure(consumer: MiddlewareConsumer) {
// 请求 ID 先行（限流响应也要带 requestId），再全局限流（docs/09）
consumer.apply(RequestIdMiddleware, GlobalRateLimitMiddleware).forRoutes('*');
}
}
===== 源文件：server/src/common/interceptors/response.interceptor.ts （第 1-30 行）=====
----- 片段说明：统一响应包装（成功码 + 数据体） -----
import { CallHandler, ExecutionContext, Injectable, NestInterceptor } from '@nestjs/common';
import { Observable } from 'rxjs';
import { map } from 'rxjs/operators';
/** 统一响应结构（docs/03） */
export interface ApiResponse<T> {
code: number;
message: string;
data: T;
requestId: string;
timestamp: number;
}
@Injectable()

```
export class ResponseInterceptor<T> implements NestInterceptor<T, ApiResponse<T>> {
intercept(context: ExecutionContext, next: CallHandler): Observable<ApiResponse<T>> {
const request = context.switchToHttp().getRequest();
const requestId: string = request.requestId ?? '';
return next.handle().pipe(
map((data) => ({
code: 0,
message: 'ok',
data: data ?? null,
requestId,
timestamp: Date.now(),
})),
);
}
}
```
===== 源文件：server/src/common/filters/all-exceptions.filter.ts （第 1-89 行）=====
----- 片段说明：全局异常过滤器：不外泄堆栈、中文业务错误、requestId 回传 -----
```
import {
ArgumentsHost,
Catch,
ExceptionFilter,
HttpException,
HttpStatus,
Logger,
} from '@nestjs/common';
import { Response } from 'express';
import {
isChineseMessage,
toChineseFallbackMessage,
type ValidationFieldError,
} from '../pipes/validation-messages';
/**
```

- 全局异常过滤器：统一错误响应结构；500 不向客户端泄露堆栈（docs/03、docs/09）

```
*
```

- 2026-10-04 中文化加固（负责人报障"英文校验文案"）：
- - 校验类 400 的 `fields[]`（字段级中文错误）**原样透传**，客户端据此把提示落到输入框下方；
- - 任何**纯英文**的 HTTP 错误消息（body-parser 的 JSON 解析失败、
- `Validation failed (uuid is expected)` 之类）一律换成中文兜底文案，
- 并把原文打 warn 供排查 —— 保证客户端永远不会看到英文原文。

```
*/
@Catch()
export class AllExceptionsFilter implements ExceptionFilter {
private readonly logger = new Logger('Exception');
catch(exception: unknown, host: ArgumentsHost) {
const ctx = host.switchToHttp();
const response = ctx.getResponse<Response>();
const request = ctx.getRequest();
const requestId: string = request.requestId ?? '';
let status = HttpStatus.INTERNAL_SERVER_ERROR;
let message = '服务器内部错误';
let errorCode: string | undefined;
let fields: ValidationFieldError[] | undefined;
```

软件名称：祁连山电竞陪玩服务平台软件 V1.0
if (exception instanceof HttpException) {
status = exception.getStatus();
const body = exception.getResponse();
if (typeof body === 'string') {
message = body;
} else if (body && typeof body === 'object') {
const msg = (body as Record<string, unknown>).message;
message = Array.isArray(msg) ? msg.join('；') : String(msg ?? exception.message);
// 业务错误码透传（如 ACCOUNT_BANNED），客户端据此做差异化处理（弹窗等）
const ec = (body as Record<string, unknown>).errorCode;
if (typeof ec === 'string') errorCode = ec;
// 校验字段明细透传（中文化在 LenientValidationPipe.exceptionFactory 里已完成）
const rawFields = (body as Record<string, unknown>).fields;
if (Array.isArray(rawFields)) {
fields = rawFields.filter((f): f is ValidationFieldError => !!f && typeof f === 'object');
}
}
// 纯英文消息 → 中文兜底（保留原文到日志）
if (!isChineseMessage(message)) {
this.logger.warn(
HTTP 错误消息为纯英文，已替换中文兜底 requestId=${requestId} method=${request.method} path=${request.path} status=${status} raw=$
{this.clip(message)},
);
message = toChineseFallbackMessage(message, status);
}
} else {
// 未预期异常：记录完整堆栈供排查，客户端只看到通用文案
this.logger.error(
未捕获异常 requestId=${requestId} method=${request.method} path=${request.path},
exception instanceof Error ? exception.stack : String(exception),
);
}
response.status(status).json({
code: status,
message,
data: null,
requestId,
timestamp: Date.now(),
...(errorCode ? { errorCode } : {}),
...(fields && fields.length > 0 ? { fields } : {}),
});
}
/** 日志里保留原文但截断，避免 SQL/堆栈把日志刷爆 */
private clip(text: string, max = 300): string {
const t = String(text ?? '');
return t.length > max ? ${t.slice(0, max)}… : t;
}
}
===== 源文件：server/src/common/guards/jwt-auth.guard.ts （第 1-288 行）=====
----- 片段说明：JWT 守卫：单设备会话、受限会话（打手态 / 充值管理员）允许清单、账号封禁实时校验 -----
import { CanActivate, ExecutionContext, ForbiddenException, Injectable, UnauthorizedException } from '@nestjs/common';
import { ConfigService } from '@nestjs/config';
import { JwtService } from '@nestjs/jwt';

```
import { InjectDataSource } from '@nestjs/typeorm';
import { DataSource } from 'typeorm';
import { AuthToken, AuthTokenKind } from '../../modules/auth/auth-token.entity';
import {
COMPANION_GONE,
COMPANION_BOSS_GONE,
checkCompanionSession,
} from '../../modules/auth/companion-session.check';
import { checkRechargeSession } from '../../modules/auth/recharge-session.check';
import {
WORKER_APP_SUNSET_GONE,
checkWorkerAppSunset,
isWorkerPrimarySession,
} from '../../modules/auth/worker-app-sunset';
export interface AuthUser {
userId: string;
role: string;
jti: string;
/** 会话种类（取自 auth_token 库中登记，不信任令牌载荷）：primary 普通登录 / companion 伴随会话 / recharge 受限充值管理员 */
kind: AuthTokenKind;
/** companion 会话所属绑定 id（primary/recharge 为 null） */
bindingId: string | null;
}
/*
```

- companion 会话失效时的中文提示（401：客户端据此清打手态并回到老板态）与**校验实现**都在
- `modules/auth/companion-session.check.ts` —— HTTP 守卫与 WS 网关共用同一份，不再各写一套
- （安全审计 F1：WS 侧曾自行其事，导致"HTTP 已 401、WS 还在推消息"）。
- 受限充值管理员会话（kind='recharge'）的校验同构，实现同样只有一份：
- `modules/auth/recharge-session.check.ts`。

```
*/
/** companion 越界访问老板侧/管理端接口时的中文提示（403：身份还在，但这个操作不允许） */
const COMPANION_FORBIDDEN = '伴随身份无此操作权限，请切回老板身份后操作';
/** 受限充值管理员越界访问非充值族接口时的中文提示（403：身份还在，但这个操作不允许） */
const RECHARGE_ADMIN_FORBIDDEN = '受限管理身份仅可处理充值审核，请用管理端完成其它操作';
/**
```

- JWT 认证守卫（docs/04）：
- 1. 校验 Bearer access token（签名/过期/type）
- 2. 校验 auth_token 表登记且未吊销（登出/轮换/踢下线立即失效）
- 3. 注入 req.user = { userId, role, jti, kind, bindingId }——主体 id 的唯一可信来源

```
*
```

- ============================== 伴随会话（companion）==============================
- 合并双端·身份切换（方案甲，负责人 2026-10-06 拍板③）：老板端内用打手账密切换身份时，
- 签发的是 `kind=companion` 的受限会话。它**不是** primary，因此既要"能用打手侧接口"，
- 又必须"绝不能被当成 primary 用"。规则如下（全部以**库中登记**为准，绝不信任令牌载荷）：

```
*
```

- A. 每请求有效性校验（每次请求都查库，无缓存）——**实现只有一份**：
- `modules/auth/companion-session.check.ts` 的 `checkCompanionSession()`，本守卫与
- `RealtimeGateway`（WS 握手 + 心跳）共用（安全审计 F1：WS 侧曾漏掉这一层：
- "HTTP 已 401，旧 WS 连接仍能 join 房间并继续收到 msg:new 全文"）。校验内容：
- - `auth_token.kind='companion'` 且 `binding_id` 非空；

- - 绑定行存在且 `status='active'`（**解绑即刻失效** → 401 中文）；
- - 绑定里的 `worker_user_id` 必须等于会话主体（防绑定 id 被张冠李戴）；
- - 打手账号未被封禁（`user.status=1`）、打手档案在职（`worker.status=1`）；
- - **老板账号未被封禁且仍存在**（`boss_user_id` → `user.status=1`，2026-10-06 缺口 1 双保险：
- 封禁/删除已对称释放绑定，这一条兜住历史悬空 active 行 → 401「老板账号已被封禁...」）。

*

- B. 接口族**白名单（默认拒绝）**——companion 只能访问：
- - `/worker/*` 打手侧全部（大厅/抢单/派单确认/开始/提交/我的订单/收益/待遇/客服会话/排行榜）
- - `/user/conversation*` 客服会话（老板/客服两段消息）
- - `/user/conversations*` 会话列表（含 type 区分 客服/老板/好友）/ 消息历史 / 发送消息 /
- **一键已读 `POST /user/conversations/:id/read`**（消息族前缀覆盖）
- - `GET /user/friends` 好友列表（打手态消息页【好友】段的**读**数据源；负责人 2026-10-06）
- - `POST /user/friends/:id/conversation` 打开与**已是好友**的会话（服务端只认 accepted 关系）
- - `GET /user/profile/*` 资料读（聊天里点头像看公开资料）
- - `/media/*` 聊天图片等媒体上传与受保护读取
- - `GET /auth/me`、`POST /auth/logout`、`POST /auth/refresh`
- 其余一律 403 中文（`COMPANION_FORBIDDEN`）。**特别地**：
- - `/user/identity/*`（绑定的绑定/解绑/状态）→ 老板侧能力，companion 不可用；
- - `/auth/password`（改密码）、`PUT /user/profile`（改资料）、`POST /user/profile/uid`（改 ID）→ 账号级写操作，companion 不可用；
- - `/user/orders`（下单）、`/user/wallet`、`/user/tips`、`/user/refunds`、`/user/complaints`、`/user/ai/*` → 老板态能力；
- - `/user/friends*` 里**除** `GET /user/friends`（列表）与 `POST /user/friends/:id/conversation`（开会话）
- 以外的全部路由 → 好友关系的**写**能力（搜索/申请/同意/拒绝/删除）与系统通知，companion 一律不可用；
- - `/admin/*`（管理端）→ 不只是白名单，`RolesGuard` 也会按 `role` 拦（companion 的 role 是 worker）。
- 白名单是**允许清单**而不是黑名单：将来新增接口默认对 companion 关闭，必须显式加进来才能用
- （黑名单模式会在"新接口忘了加规则"时静默放开，这是不可接受的默认值）。

*

- C. 与单设备登录（primary）的关系：
- - primary 的校验路径**逐字节未改**（含 revoked=1/2 的原有文案）；
- - **各类会话互不干扰**（`AuthService.issueTokens` 的实际实现）： primary 登录/续期**只顶掉同账号的其它
- primary**（`WHERE user_id=? AND kind='primary'`：登录 replaced → revoked=2「账号已在其他设备登录」，
- 续期 → revoked=1），**不触碰 companion/recharge**；companion 签发/续期**只顶同账号的其它 companion**，

- recharge 同理 —— 都绝不触碰 primary。
- - **单设备铁律一点没松**：primary ↔ primary 仍然互斥（同账号第二次登录照旧顶掉第一次，文案不变）——
- 在只有 primary 行的世界里，带 `kind='primary'` 的吊销语句与历史实现**逐字节等价**（过滤条件恒真）。
- companion 侧另有三重约束兜底：12h TTL + 每请求校验绑定 active + 解绑/封禁即吊销。

*

- ========== 受限充值管理员会话（recharge，2026-10-06）==========
- 老板端「身份」里新增的**管理员态**（负责人定案：最多 5 个账号、权限仅限积分/充值相关）。
- 它与 companion 是**同一类东西的两个实例**——"受限会话"：复用同一套机制，不新造一套。

*

- A. 每请求有效性校验（无缓存，实现只有一份 `checkRechargeSession`）：
- - 会话主体仍在 `recharge_admin` 白名单（**移出白名单 → 下一次请求立即 401 中文**）；
- - 账号未封禁（`user.status=1`）、仍是 `role='admin'`。
- B. 接口族**白名单（默认拒绝）**——recharge 只能访问：
- - `/recharge/*`　　充值审核族（待审列表 / 发放 / 驳回 / 当前身份 / 凭证读取）
- - `GET /auth/me`、`POST /auth/logout`、`POST /auth/refresh`
- 其余一律 403 中文（`RECHARGE_ADMIN_FORBIDDEN`）。**特别地**：
- - `/admin/*`（含 `/admin/users`、`/admin/products`、`/admin/recharge/*`）→ 403。
- 为什么连本功能的**管理端**接口也不放行：那些接口的定义是"管理端（`admin:*` 权限）"，
- 受限会话要改配置/收款码必须走管理端登录（primary），这正是"权限仅限积分相关"的落地。

- - /user/*（/user/orders、/user/wallet、/user/identity/*、/user/recharge/*、PUT /user/profile…）→ 403；
- - /auth/password（改密码）→ 403；
- - /media/* → 403（**刻意不给**：/media/protected/:id 对 role=admin 放行**任意**受保护媒体，
- 受限会话不该拿到这个面。充值凭证改走 /recharge/proof/:requestId——只放行"这张申请的那张图"）。
- 与 companion 一样是**允许清单**：将来新增接口默认对 recharge 关闭。
- C. WS：RealtimeGateway **拒绝** recharge 会话握手（受限会话无实时需求；且它的 role 是 admin，
- 放进来会 join 管理端广播房间 admin —— 那是明确的越权面）。

*

- ==========工作端到期收口（2026-10-06 定案）==========
- 到期后（system_setting.worker_app_sunset_at，默认 2026-10-13 00:00 Asia/Shanghai）：
- - **拒绝** role=worker 的 **primary** 会话：本守卫每请求 401 中文
- （/worker/*、/user/* 及一切受保护接口），AuthService.login 403 中文，
- RealtimeGateway 握手/心跳拒绝 —— 三处**共用同一个判定函数**
- modules/auth/worker-app-sunset.ts 的 checkWorkerAppSunset()（读库无缓存）。
- - **绝不影响**：老板端「打手态」companion 会话（kind='companion'，role 同样是 worker，
- 但它正是工作端的替代品，必须照常可用）、老板端（user）、管理端（admin）、受限充值会话（recharge）。
- 边界由 isWorkerPrimarySession(kind，role) 一句话定义（**只看 kind=primary**），
- 守卫与网关共用同一句，不各写一遍。

```
*/
@Injectable()
export class JwtAuthGuard implements CanActivate {
constructor(
private readonly jwt: JwtService,
private readonly config: ConfigService,
@InjectDataSource() private readonly dataSource: DataSource,
) {}
async canActivate(context: ExecutionContext): Promise<boolean> {
const request = context.switchToHttp().getRequest();
const token = this.extractBearer(request);
if (!token) {
throw new UnauthorizedException('未登录');
}
let payload: { sub?: string; role?: string; jti?: string; type?: string };
try {
payload = await this.jwt.verifyAsync(token, {
secret: this.config.get<string>('JWT_ACCESS_SECRET'),
});
} catch {
throw new UnauthorizedException('登录已过期，请重新登录');
}
if (payload.type !== 'access' || !payload.sub || !payload.jti) {
throw new UnauthorizedException('令牌类型错误');
}
// 会话登记校验（登出/轮换后立即失效；被新设备登录顶掉则明确提示，docs/04 单设备会话）
const session = await this.dataSource.getRepository(AuthToken).findOne({
where: { jti: payload.jti },
});
if (!session) {
throw new UnauthorizedException('登录已失效，请重新登录');
}
```

```
// 会话种类以库中登记为准（令牌载荷里的 kind 只作观测，不参与判定）
const kind: AuthTokenKind =
session.kind === 'companion' ? 'companion' : session.kind === 'recharge' ? 'recharge' : 'primary';
if (session.revoked === 1) {
throw new UnauthorizedException(
kind === 'primary'
? '登录已失效，请重新登录'
: kind === 'companion'
? COMPANION_GONE
: '受限管理身份已失效，请重新绑定管理员账号',
);
}
if (session.revoked === 2) {
throw new UnauthorizedException(
kind === 'primary'
? '账号已在其他设备登录'
: kind === 'companion'
? '伴随身份已失效（该打手账号已在别处登录），请重新绑定'
: '受限管理身份已失效（该管理员账号已在别处登录），请重新绑定',
);
}
// 主体绑定：令牌 sub 必须与会话登记的用户一致。
// 仅校验「jti 存在」是不够的——签名密钥一旦泄露，攻击者可拿任一有效 jti
// 搭配任意 sub 伪造身份（实测可冒用他人）。此处把身份钉死在会话记录上。
if (String(session.userId) !== String(payload.sub)) {
throw new UnauthorizedException('登录已失效，请重新登录');
}
// 会话寿命校验：令牌不得超出会话登记的到期时间。
// 否则签名密钥泄露时，可用截获的 jti 铸造一张超长期令牌，
// 在原令牌 2 小时过期后仍持续有效（实测可复现）。
if (session.expireAt && new Date(session.expireAt).getTime() < Date.now()) {
throw new UnauthorizedException(
kind === 'primary'
? '登录已过期，请重新登录'
: kind === 'companion'
? COMPANION_GONE
: '受限管理身份已过期，请重新绑定管理员账号',
);
}
// companion 专属：绑定有效性（每请求实时查库） + 接口族白名单
if (kind === 'companion') {
// 校验实现不在本文件：与 WS 网关共用 checkCompanionSession（见该文件顶部说明）
const check = await checkCompanionSession(this.dataSource, {
bindingId: session.bindingId,
userId: String(session.userId),
});
if (!check.ok) {
throw new UnauthorizedException(check.message);
}
if (!isCcompanionAllowedRoute(request.method, request.path)) {
```

throw new ForbiddenException(COMPANION_FORBIDDEN);
}
}
// recharge 专属（受限充值管理员）：白名单/封禁有效性（每请求实时查库）+ 接口族白名单
if (kind === 'recharge') {
// 校验实现同样只有一份：`checkRechargeSession`（HTTP 守卫与 WS 网关共用）
const check = await checkRechargeSession(this.dataSource, { userId: String(session.userId) });
if (!check.ok) {
// **移出白名单 / 账号被封禁 → 立即 401 中文**（不等令牌过期，也不需要重新登录）
throw new UnauthorizedException(check.message);
}
if (!isRechargeAdminAllowedRoute(request.method, request.path)) {
throw new ForbiddenException(RECHARGE_ADMIN_FORBIDDEN);
}
}
/**

- ========== 工作端到期收口（2026-10-06 定案）==========
- 到期后（`system_setting.worker_app_sunset_at`，默认 `2026-10-13 00:00` Asia/Shanghai）
- **worker 的 primary 会话**一律 401 中文：`/worker/*`、`/user/*` 以及其余全部受保护接口。

*

- 判定**不在本文件**：读配置 + 判断都在 `modules/auth/worker-app-sunset.ts`
- （与 `AuthService.login`、`RealtimeGateway` 共用同一函数；安全审计 F1 的教训：
- 同一规则写两份必然分叉）。本处只负责"哪种会话受影响"这一句：
- **必须同时满足 `kind='primary'` 且 `role='worker'`**，
- 因为老板端「打手态」的 companion 会话 `role` **也是 worker** ——
- 它正是工作端的替代品，**必须照常可用**（本次最重要的一条不变量）。
- 老板端（user）/ 管理端（admin）/ 受限充值会话（recharge）同样一概不受影响。

*/
if (isWorkerPrimarySession(kind, session.role || payload.role)) {
const sunset = await checkWorkerAppSunset(this.dataSource);
if (sunset.sunset) {
throw new UnauthorizedException(WORKER_APP_SUNSET_GONE);
}
}
request.user = {
userId: String(session.userId),
// 角色同样以会话登记为准，不信任令牌载荷（防自签提权）
role: session.role || payload.role || '',
jti: payload.jti,
kind,
bindingId: session.bindingId ? String(session.bindingId) : null,
} satisfies AuthUser;
return true;
}
private extractBearer(request: { headers?: Record<string, unknown> }): string | null {
const auth = request.headers?.['authorization'];
if (typeof auth === 'string' && auth.startsWith('Bearer ')) {
return auth.slice(7);
}
return null;

}
}
/**

- companion 接口族白名单（纯函数，便于单测/复查）。

*

- 入参 path 为 Express 的 request.path（含全局前缀 /api/v1），这里先剥掉前缀再比对，
- 因此路由前缀变更（如 /api/v2）不会让白名单静默失效。

*/
export function isCompanionAllowedRoute(method: string, path: string): boolean {
const m = String(method ?? 'GET').toUpperCase();
const p = String(path ?? '').replace(/^\/api\/v\d+/, '') || '/';
// 打手侧全部接口（大厅/抢单/派单确认/开始/提交/我的订单/收益/待遇/客服/排行榜）
if (p === '/worker' || p.startsWith('/worker/')) return true;
// 消息：客服会话（老板/客服两段）与会话消息读写；打手态消息页的唯一数据源
// - POST /user/conversations/:id/read（一键已读 REST 兜底，2026-10-06 缺口 3）由下面的
// /user/conversations/* 前缀规则**一并放行**（打手态也要能一键已读 → 显式记录在案，避免被误删）
if (p === '/user/conversation' || p.startsWith('/user/conversation/')) return true;
if (p === '/user/conversations' || p.startsWith('/user/conversations/')) return true;
/**

- 好友（打手态消息页的【好友】段，负责人 2026-10-06）：**只放行两条读/开会话路径。**

*
===== 源文件：server/src/modules/auth/auth.controller.ts （第 1-190 行）=====
----- 片段说明：认证接口：注册、登录、刷新、登出、当前主体 -----
import { Body, Controller, Get, HttpCode, NotFoundException, Post, Req, UseGuards } from '@nestjs/common';
import { ApiBearerAuth, ApiOperation, ApiTags } from '@nestjs/swagger';
import { ConfigService } from '@nestjs/config';
import { Request } from 'express';
import { clientIpOf } from '../../common/client-ip';
import { CurrentUser } from '../../common/decorators/current-user.decorator';
import { JwtAuthGuard } from '../../common/guards/jwt-auth.guard';
import { RealtimeGateway } from '../../realtime/realtime.gateway';
import { AuthService } from './auth.service';
import { EmailCodeService } from './email-code.service';
import {
LoginDto,
LogoutDto,
RefreshDto,
RegisterDto,
ResetPasswordDto,
SendEmailCodeDto,
} from './dto/auth.dto';
@ApiTags('auth')
@Controller('auth')
export class AuthController {
constructor(
private readonly authService: AuthService,
/** RealtimeModule 是 @Global（与 RechargeService / BindingReleaseService 同一注入方式）：登出后主动断开 WS */
private readonly realtime: RealtimeGateway,
) {}
/** 公开：老板端注册（工作端/管理员账号由后台创建，docs/04） */
@Post('register')
@ApiOperation({ summary: '公开：老板端注册（可选邮箱+邮箱验证码；不传 email 时行为与旧版一致）' })

```
register(@Body() dto: RegisterDto) {
return this.authService.register(dto);
}
/**
```

- 公开：获取邮箱验证码（负责人 2026-10-05：邮箱验证替代短信验证）
- 限流：同邮箱 60s / 同邮箱 5 次·小时 / 同 IP 10 次·小时；响应**不回显验证码**

```
*/
@Post('email/code')
@HttpCode(200)
@ApiOperation({ summary: '公开：获取邮箱验证码（scene: register/reset；防枚举；响应不回显验证码）' })
sendEmailCode(@Body() dto: SendEmailCodeDto, @Req() req: Request) {
return this.authService.sendEmailCode(dto, this.clientIp(req), req.header('user-agent') ?? '');
}
/** 公开：邮箱验证码找回密码（不依赖旧密码；成功后吊销该账号全部会话，需重新登录） */
@Post('password/reset')
@HttpCode(200)
@ApiOperation({ summary: '公开：邮箱验证码重置密码（成功后吊销该用户全部 access/refresh 会话）' })
resetPassword(@Body() dto: ResetPasswordDto, @Req() req: Request) {
return this.authService.resetPasswordByEmail(dto, this.clientIp(req), req.header('user-agent') ?? '');
}
/** 公开：三端统一登录入口，按 role 分流校验（docs/04） */
@Post('login')
@HttpCode(200)
@ApiOperation({ summary: '公开：登录（role: user/worker/admin，与实际账号不符 403，限流 5 次/分/IP+账号）' })
login(@Body() dto: LoginDto, @Req() req: Request) {
return this.authService.login(dto, this.clientIp(req), req.header('user-agent') ?? '');
}
/** 公开：refresh 轮换（docs/04） */
@Post('refresh')
@HttpCode(200)
@ApiOperation({ summary: '公开：刷新令牌（轮换制，旧 refresh 即吊销）' })
refresh(@Body() dto: RefreshDto, @Req() req: Request) {
return this.authService.refresh(dto.refreshToken, this.clientIp(req));
}
/** 隧道回源（回环）时以 CF-Connecting-IP 还原真实客户端 IP；直连时不信任该头（防伪造绕过限流）
```

- —— 2026-10-06 抽到 `common/client-ip.ts`（身份绑定接口用同一套判定，限流键必须一致） */

```
private clientIp(req: Request): string {
return clientIpOf(req);
}
/**
```

- 登出（docs/04）：**把这次登录的全部令牌（access + refresh）一次吊销。**

```
*
```

- - `userId` 由守卫从**库中会话登记行**取出（`request.user.userId`），用它把该账号该族的活行收干净；
- 客户端**不传** `refreshToken` 也保证退干净（旧实现漏的正是这一条：实测残留 `revoked=0` 的 refresh 行）；
- - `kind` 同样以库中登记为准 ⇒ companion / recharge 登出**绝不触碰 primary**；
- - `revokedJtis` 交给 `RealtimeGateway.disconnectByJti()` 主动断开这些会话已建立的 WS
- （新增行为，默认事件与 companion/recharge 释放路径一致）。

```
*/
@Post('logout')
@HttpCode(200)
```

```
@UseGuards(JwtAuthGuard)
@ApiBearerAuth()
@ApiOperation({ summary: '登出（吊销本次登录的全部会话行：access + refresh；三类会话同一规则）' })
async logout(@Body() dto: LogoutDto, @CurrentUser() user: { userId: string; role: string; jti: string }) {
const r = await this.authService.logout(user.jti, dto.refreshToken, user.userId);
// 登出即吊销 → 已建立的 WS 必须主动断开（F1 同类根因：只改库不踢线 = 旧连接继续收消息）
this.realtime.disconnectByJti(r.revokedJtis, 'auth:expired');
return { ok: r.ok };
}
@Post('password')
@HttpCode(200)
@UseGuards(JwtAuthGuard)
@ApiBearerAuth()
@ApiOperation({ summary: '修改密码（校验旧密码；改密后全部会话吊销需重新登录）' })
changePassword(@Body() dto: { oldPassword: string; newPassword: string }, @CurrentUser() user: { userId: string }) {
return this.authService.changePassword(user.userId, dto);
}
@Get('me')
@UseGuards(JwtAuthGuard)
@ApiBearerAuth()
@ApiOperation({ summary: '当前登录主体信息（worker 附权限位、admin 附角色与权限点）' })
me(@CurrentUser() user: { userId: string; role: string }) {
return this.authService.me(user);
}
}
/**
```

- **仅自动化测试用**的邮箱验证码检查端点（父任务要求「补一条测试用注入点」）。

*

- 安全设计（为什么不能把 verify 做成公开接口）：
- -「校验正确码 → 一次性作废」意味着**任何会消费验证码的公开接口都是攻击面**：
- 若 `POST /auth/email/verify` 公开，攻击者只要猜到/试到码就能在真正注册前把码烧掉；
- 更糟的是它与 register/reset 争抢同一份一次性凭据，会产生难以推理的竞态。
- - 因此生产接口只保留 **email/code（发码）+ register / password/reset（内部验码）**，
- 校验逻辑作为**内部能力**放在 EmailCodeService，由业务方法调用。
- - 本控制器仅用于测试断言「生成/校验/过期/一次性/错满 5 次」这些**内部规则**：
- 每条路由先调 `ensureTestMode()`，**任何非 `MAIL_DRIVER=console` 的部署一律 404**（生产不可达）。

```
*/
@ApiTags('auth-test')
@Controller('auth/email/_test')
export class AuthEmailTestController {
constructor(
private readonly emailCode: EmailCodeService,
private readonly config: ConfigService,
) {}
/**
```

- 生产守卫（**唯一一把钥匙**）：必须显式 `MAIL_ALLOW_TEST_ENDPOINTS=true`，否则一律 404。

*

- 为什么不再要求 MAIL_DRIVER=console：真实通道端到端测试需要在 **smtp 驱动（真发信）** 下读取
- Redis 里的验证码来做断言（生产接口绝不回显验证码），若同时要求 console 驱动就无法两全。
- 生产只需保证这一项为 false（默认即 false；.env.example 里也写明生产必须 false）。

软件名称：祁连山电竞陪玩服务平台软件 V1.0
*/
private ensureTestMode(): void {
if (String(this.config.get<string>('MAIL_ALLOW_TEST_ENDPOINTS', 'false')).toLowerCase() !== 'true') {
throw new NotFoundException();
}
}
/** 读取 Redis 里当前有效码（仅测试；生产 404） */
@Get('peek')
@ApiOperation({ summary: '【测试专用】查看当前有效验证码（非 console 驱动返回 404）' })
async peek(@Req() req: Request) {
this.ensureTestMode();
const email = String(req.query.email ?? '').trim().toLowerCase();
const scene = String(req.query.scene ?? '');
if (!email || (scene !== 'register' && scene !== 'reset')) {
throw new NotFoundException();
}
const { code, expired } = await this.emailCode.describeCodeForTest(email, scene);
return { email, scene, code, expired };
}
/** 校验验证码（仅测试；生产 404）。返回判定结果，不抛业务异常，便于断言各分支 */
@Post('verify')
@HttpCode(200)
@ApiOperation({ summary: '【测试专用】校验验证码（非 console 驱动返回 404）' })
async verify(@Body() body: { email?: string; scene?: string; code?: string }) {
this.ensureTestMode();
const email = String(body?.email ?? '').trim().toLowerCase();
const scene = String(body?.scene ?? '');
if (!email || (scene !== 'register' && scene !== 'reset')) {
throw new NotFoundException();
}
const outcome = await this.emailCode.verifyCode(email, scene, String(body?.code ?? ''));
return outcome.ok ? { ok: true } : { ok: false, reason: outcome.reason };
}
/** 开关「把发信结果打桩为失败」（仅测试；生产 404） */
@Post('fail-mode')
@HttpCode(200)
@ApiOperation({ summary: '【测试专用】打桩发信失败（非 console 驱动返回 404）' })
async failMode(@Body() body: { on?: boolean; scope?: string }) {
this.ensureTestMode();
const on = body?.on === true;
const scope = String(body?.scope ?? 'smtp');
await this.emailCode.setForceFail(on);
return { ok: true, forceFail: on, scope };
}
}
===== 源文件：server/src/modules/auth/auth.service.ts （第 26-172 行）=====
----- 片段说明：认证服务：令牌 TTL 常量、会话种类归一化、注册 -----
const ACCESS_TTL_SECONDS = 2 * 60 * 60; // 2 小时
const REFRESH_TTL_SECONDS = 30 * 24 * 60 * 60; // 30 天（轮换制）
const LOGIN_RATE_LIMIT = 5; // 5 次/分/IP+账号（docs/03）
export interface TokenPair {
accessToken: string;

refreshToken: string;
expiresIn: number;
}
/** 签发选项：kind≠primary 时携带所属绑定 id 与更短 TTL（见 worker-binding-limit.ts / recharge-config.ts） */
export interface IssueTokenOptions {
kind?: AuthTokenKind;
bindingId?: string | null;
}
/**

- 会话种类归一化：**以 opts.kind 为准，未知值一律回落 primary**（不信任调用方传进来的字符串）。
- 抽成函数是为了让 `issueTokens` 与 `refresh` 的判定**永远同源**（两处各写一遍三元表达式必然分叉）。

*/
function normalizeKind(kind: unknown): AuthTokenKind {
return kind === 'companion' || kind === 'recharge' ? kind : 'primary';
}
/**

- 认证服务（docs/04）：
- - 登录失败统一返回"账号或密码错误"，不泄露账号是否存在
- - 每次登录/刷新登记 auth_token，登出/轮换即吊销
- - 登录限流 + 全量登录日志

*/
@Injectable()
export class AuthService {
private readonly logger = new Logger('Auth');
constructor(
@InjectRepository(User) private readonly userRepo: Repository<User>,
@InjectRepository(AuthToken) private readonly authTokenRepo: Repository<AuthToken>,
@InjectRepository(LoginLog) private readonly loginLogRepo: Repository<LoginLog>,
@InjectRepository(Worker) private readonly workerRepo: Repository<Worker>,
@InjectRepository(Admin) private readonly adminRepo: Repository<Admin>,
@InjectRepository(AdminRole) private readonly adminRoleRepo: Repository<AdminRole>,
@InjectDataSource() private readonly dataSource: DataSource,
private readonly jwt: JwtService,
private readonly config: ConfigService,
private readonly rateLimit: RateLimitService,
private readonly walletService: WalletService,
/** 分部接单权限（/auth/me 附带展示，判定仍在接单侧实时查库） */
private readonly divisionPerm: WorkerDivisionPermissionService,
/** 邮箱验证码（负责人 2026-10-05：邮箱验证替代短信验证） */
private readonly emailCode: EmailCodeService,
/** 邮件发送（失败只 warn + 返回 false，绝不抛给业务层） */
private readonly mailer: MailerService,
/** 找回密码审计留痕（docs/09） */
private readonly opLog: OperationLogService,
) {}
/**

- 老板端自助注册（工作端/管理员账号由后台创建，docs/04；可选自定义展示 ID，支持中文）

*

- 邮箱扩展（负责人 2026-10-05）：可选 `email` + `emailCode`。
- - **不传 email**：行为与老版本**逐字节一致**（老客户端零影响）；

- - 传 email：必须带有效 emailCode，否则中文 400；
- - 邮箱唯一性先于验码检查（不浪费用户验证码，也不让已占用邮箱消耗限流额度）。

```
*/
async register(dto: {
username: string;
password: string;
nickname?: string;
uid?: string;
email?: string;
emailCode?: string;
}) {
const exists = await this.userRepo.findOne({ where: { username: dto.username } });
if (exists) {
throw new ConflictException('用户名已存在');
}
const email = this.normalizeEmail(dto.email);
if (email) {
if (!dto.emailCode) {
throw new BadRequestException('请填写邮箱验证码');
}
const taken = await this.userRepo.findOne({ where: { email } });
if (taken) {
throw new ConflictException('该邮箱已被注册');
}
}
if (dto.uid) {
// 2026-09-25：注册即校验 ID 格式（2-20 位中文/字母/数字/下划线），与好友搜索/修改 ID 规则一致
if (!UID_PATTERN.test(dto.uid)) {
throw new BadRequestException('ID 格式非法（2-20 位中文/字母/数字/下划线）');
}
const uidExists = await this.userRepo.findOne({ where: { uid: dto.uid } });
if (uidExists) {
throw new ConflictException('该 ID 已被使用');
}
}
// 验码（一次性：成功即作废；错 ≥5 次作废；过期中文提示）
if (email) {
await this.emailCode.verifyOrThrow(email, 'register', dto.emailCode!);
}
const passwordHash = await bcrypt.hash(dto.password, 10);
const verifiedAt = email ? new Date() : null;
// 注册 = 账号 + 钱包（事务，docs/06：一人一钱包，注册即建）
let user: User;
try {
user = await this.dataSource.transaction(async (em) => {
const u = await em.save(
em.create(User, {
username: dto.username,
uid: dto.uid ?? (await this.generateUid()),
passwordHash,
```

nickname: dto.nickname || dto.username,
role: 'user',
status: 1,
email,
emailVerifiedAt: verifiedAt,
}),
);
await this.walletService.createWalletInTx(em, u.id);
return u;
});
} catch (e) {
// 并发兜底：唯一索引 UQ_user_email / 用户名唯一约束被同时注册的请求撞到 → 中文冲突而不是 500
if (isDuplicateKeyError(e)) {
throw new ConflictException(email ? '该邮箱已被注册' : '用户名已存在');
}
throw e;
}
return {
id: user.id,
username: user.username,
uid: user.uid,
nickname: user.nickname,
// 仅回显是否已绑定邮箱，**不回显验证码**
email: user.email,
emailVerified: user.emailVerifiedAt !== null,
};
}
/**

- 获取邮箱验证码（负责人 2026-10-05）

*

- - scene=register：邮箱已被注册 → 中文 400（注册场景本就要告知占用，无法也不应隐藏）
- - scene=reset：**防用户枚举** —— 邮箱未注册时**仍返回成功**（文案与真发一致），但不发信、不占额度；
- 内部只记一条 warn 供运维排查。
- - 响应**绝不返回验证码**，只返回「已发送」。

*/
===== 源文件：server/src/modules/auth/auth.service.ts （第 250-396 行）=====
----- 片段说明：认证服务：邮箱验证码重置、登录（统一失败文案）、封禁校验 -----
async resetPasswordByEmail(
dto: { email: string; emailCode: string; newPassword: string },
ip: string,
/** 请求来源 UA（审计用；不传时该列留空） */
userAgent = '',
) {
// 密码规则沿用既有校验（8-64 位；DTO 已拦一层，这里兜业务层直接调用）
if (!dto.newPassword || dto.newPassword.length < 8 || dto.newPassword.length > 64) {
throw new BadRequestException('新密码须为 8-64 位');
}
const email = this.normalizeEmail(dto.email);
if (!email) throw new BadRequestException('邮箱格式不正确');
const user = await this.userRepo.findOne({ where: { email } });
// 枚举防护：未注册邮箱也走「验证码已过期」这条通用文案（不暴露邮箱是否存在）
if (!user) {

```
await this.writeLoginLog('user', null, email, ip, '', 'fail', '找回密码：邮箱未注册');
this.logger.warn(找回密码：邮箱未注册 email=${maskEmail(email)});
throw new BadRequestException('验证码已过期，请重新获取');
}
// 验码：一次性作废；错 ≥5 次作废；过期中文提示
await this.emailCode.verifyOrThrow(email, 'reset', dto.emailCode);
await this.userRepo.update(user.id, {
passwordHash: await bcrypt.hash(dto.newPassword, 10),
emailVerifiedAt: new Date(),
});
// 改密后全部会话吊销（与既有 changePassword 完全一致：防旧设备继续在线）
const revoked = await this.authTokenRepo.update({ userId: user.id, revoked: 0 }, { revoked: 1 });
// 审计（docs/09）：不落密码/验证码，只落主体与动作
this.opLog.log({
adminUserId: String(user.id),
adminName: user.nickname ?? user.username,
module: 'auth',
action: 'password_reset_by_email',
method: 'POST',
path: '/auth/password/reset',
requestBody: JSON.stringify({ userId: String(user.id), email: maskEmail(email), via: 'email_code' }),
ip,
result: 'success',
errorMsg: null,
});
await this.writeLoginLog('user', user.id, user.username, ip, userAgent, 'success', '邮箱验证码重置密码');
this.logger.log(
找回密成功 userId=${user.id} email=${maskEmail(email)} 吊销会话=${revoked.affected ?? 0},
);
return { ok: true, message: '密码已重置，请使用新密码登录' };
}
async login(dto: { username: string; password: string; role: string }, ip: string, userAgent: string) {
const requestKey = login:${ip}:${dto.username};
if (!(await this.rateLimit.allow(requestKey, LOGIN_RATE_LIMIT, 60))) {
await this.writeLoginLog(dto.role, null, dto.username, ip, userAgent, 'fail', '触发登录限流');
throw new ForbiddenException('登录尝试过于频繁，请稍后再试');
}
/**
```

- 工作端到期收口（负责人 2026-10-06 定案「彻底废除工作端」）：
- `system_setting.worker_app_sunset_at` 到期后，**工作端登录直接 403 中文。**

*

- - 判定**只有一份**：`modules/auth/worker-app-sunset.ts`（读配置 + 判断），
- 与 `JwtAuthGuard`（已登录的 worker primary → 401）、`RealtimeGateway`（WS 拒连）共用
- —— 不在本文件里复制第二份 SQL 或第二个日期比较（安全审计 F1 的同类教训）。
- - 位置刻意在**凭据校验之前**：工作端 App 一打开就登录，越早给出迁移文案越好；
- 这里不泄露任何账号信息（限流先行、登录日志照写、文案对未登录者本就是公开口径）。
- - **只挡 `role=worker` 的登录**：老板端（user）/ 管理端（admin）不受影响；
- 老板端「打手态」走 `POST /user/identity/bind`（companion 会话）**不经过本方法**，
- 因此替代品照常可用（本次最重要的一条不变量）。

*/

```
if (dto.role === 'worker') {
const sunset = await checkWorkerAppSunset(this.dataSource);
if (sunset.sunset) {
await this.writeLoginLog(dto.role, null, dto.username, ip, userAgent, 'fail', '工作端已停止服务（到期收口）');
this.logger.log(拒绝工作端登录（已到期 ${sunset.text}）username=${dto.username});
throw new ForbiddenException(WORKER_APP_SUNSET_GONE);
}
}
const user = await this.userRepo
.createQueryBuilder('u')
.addSelect('u.passwordHash')
.where('u.username = :username', { username: dto.username })
.getOne();
// 统一错误文案：不区分账号不存在/密码错误（docs/03）
const fail = async (reason: string) => {
await this.writeLoginLog(dto.role, user?.id ?? null, dto.username, ip, userAgent, 'fail', reason);
throw new UnauthorizedException('账号或密码错误');
};
if (!user || !user.passwordHash) {
await fail('账号不存在');
}
if (user!.role !== dto.role) {
await this.writeLoginLog(dto.role, user!.id, dto.username, ip, userAgent, 'fail', '角色不符');
throw new ForbiddenException('账号角色不符');
}
// 封禁检查（负责人需求）：到期自动解封继续登录；封禁中返回 errorCode=ACCOUNT_BANNED（客户端弹窗展示结束时间）
// 2026-10-06 抽成公共方法（逻辑逐字未改）：绑定打手账号时复用同一套封禁语义
await this.assertNotBanned(user!, dto.role, dto.username, ip, userAgent);
const ok = await bcrypt.compare(dto.password, user!.passwordHash);
if (!ok) {
await fail('密码错误');
}
const tokens = await this.issueTokens(user!);
await this.userRepo.update(user!.id, { lastLoginAt: new Date(), lastLoginIp: ip });
await this.writeLoginLog(dto.role, user!.id, dto.username, ip, userAgent, 'success', null);
this.logger.log(登录成功 userId=${user!.id} role=${dto.role});
return tokens;
}
/**
```

- 封禁/停用校验（**从 login 逐字抽出，行为完全不变**）：
- 到期自动解封继续流程；封禁中抛 403 + `errorCode=ACCOUNT_BANNED`（带结束时间）。

*

- 抽出目的（合并双端·身份切换）：老板在 `POST /user/identity/bind` 里绑定打手账号时，
- 必须与打手自己登录时**同一套**封禁判定，绝不新写第二套。

```
*/
async assertNotBanned(user: User, role: string, loginName: string, ip: string, userAgent: string): Promise<void> {
if (user.status === 1) return;
const bannedUntil = user.bannedUntil;
if (bannedUntil && bannedUntil.getTime() <= Date.now()) {
// 封禁到期：自动解封，继续正常流程
```

```
await this.userRepo.update(user.id, { status: 1, bannedUntil: null });
return;
}
await this.writeLoginLog(role, user.id, loginName, ip, userAgent, 'fail', '账号被封禁');
throw new HttpException(
{
message: bannedUntil
? 您的账号已被封禁，结束时间：${formatChinaTime(bannedUntil)}
: '您的账号已被永久封禁',
errorCode: 'ACCOUNT_BANNED',
},
HttpStatus.FORBIDDEN,
);
}
/**
```

- 密码校验（bcrypt.compare）—— 登录与「绑定打手身份」**共用同一实现。**
- 兼容旧账号（无 passwordHash）→ 一律 false，不抛异常。

```
*/
===== 源文件：server/src/modules/auth/auth.service.ts （第 402-531 行）=====
----- 片段说明：认证服务：身份事件留痕、refresh 轮换 -----
/** 登录/绑定审计留痕（复用 login_log 表；userType 记 'identity' 便于区分"登录"与"绑定"事件） */
recordIdentityEvent(
userId: string | null,
username: string,
ip: string,
userAgent: string,
result: 'success' | 'fail',
reason: string | null,
): Promise<LoginLog> {
return this.writeLoginLog('identity', userId, username, ip, userAgent, result, reason);
}
/**
```

- refresh 轮换：吊销旧 refresh，签发新对（docs/04）

```
*
```

- 伴随会话（2026-10-06 合并双端）： companion 的 refresh **也在同一入口轮换**，但:
- - 轮换前校验绑定仍 active（解绑后即使 refresh 没过期也拒绝续期）；
- - 续签只顶该账号的其它 companion，**绝不动 primary**（primary 路径逐字节不变）；
- - 续签 TTL 仍按 companion（12h 硬上限），不会把 12h 续成 30 天。

```
*
```

- @param opts.requireCompanion 安全审计 F5：`POST /user/identity/refresh`（文档口径"只吃
- companion refreshToken"的公开接口）传 true —— **库中登记不是 companion 的令牌一律 401 中文，
- 且在任何写操作之前返回**（旧实现会把老板 primary 的 refreshToken 当 companion 续期，
- 顺带走 primary 轮换分支把该账号正在用的 primary 会话吊销掉，实测原 token 立刻 401）。
- @param opts.requireRecharge 同一条闸门的充值版：`POST /user/identity/refresh-recharge-admin` 传 true
- —— 非 recharge 令牌（含 primary/companion）一律 401 中文，**在任何写操作之前返回**，
- 绝不让别的会话借这个入口被轮换/吊销。

```
*/
async refresh(refreshToken: string, ip: string, opts?: { requireCcompanion?: boolean; requireRecharge?: boolean }) {
let payload: { sub?: string; jti?: string; type?: string; role?: string };
try {
payload = await this.jwt.verifyAsync(refreshToken, {
secret: this.config.get<string>('JWT_REFRESH_SECRET'),
```

```
});
} catch {
throw new UnauthorizedException('登录已过期，请重新登录');
}
if (payload.type !== 'refresh' || !payload.sub || !payload.jti) {
throw new UnauthorizedException('令牌类型错误');
}
const session = await this.authTokenRepo.findOne({ where: { jti: payload.jti } });
// 会话种类以库中登记为准（不信任令牌载荷，防自签提权/越权续期）
const kind: AuthTokenKind = normalizeKind(session?.kind);
// F5：kind 闸门放在最前面（早于任何 update/吊销语句）——非 companion 令牌从这里出去，
// 不可能对 primary 会话产生任何副作用。
if (opts?.requireCompanion && kind !== 'companion') {
throw new UnauthorizedException(COMPANION_REFRESH_ONLY);
}
if (opts?.requireRecharge && kind !== 'recharge') {
throw new UnauthorizedException(RECHARGE_REFRESH_ONLY);
}
if (!session || session.revoked === 1) {
throw new UnauthorizedException(
kind === 'primary'
? '登录已失效，请重新登录'
: kind === 'companion'
? '伴随身份已失效，请重新绑定打手账号'
: '受限管理身份已失效，请重新绑定管理员账号',
);
}
if (session.revoked === 2) {
throw new UnauthorizedException(
kind === 'primary'
? '账号已在其他设备登录'
: kind === 'companion'
? '伴随身份已失效（该打手账号已在别处登录），请重新绑定'
: '受限管理身份已失效（该管理员账号已在别处登录），请重新绑定',
);
}
// 主体绑定：refresh 令牌的 sub 必须与会话登记的用户一致。
// 否则签名密钥泄露后，可用自己的有效 jti 搭配他人 sub 铸造出他人的全新会话。
if (String(session.userId) !== String(payload.sub)) {
throw new UnauthorizedException('登录已失效，请重新登录');
}
// 会话寿命：refresh 不得超出会话登记到期时间（防用截获 jti 无限续期）
if (session.expireAt && new Date(session.expireAt).getTime() < Date.now()) {
throw new UnauthorizedException(
kind === 'primary'
? '登录已过期，请重新登录'
: kind === 'companion'
? '伴随身份已过期，请重新绑定打手账号'
: '受限管理身份已过期，请重新绑定管理员账号',
);
```

```
}
const user = await this.userRepo.findOne({ where: { id: session.userId } });
if (!user || user.status !== 1) {
throw new UnauthorizedException('账号不可用');
}
// companion：每次续期都校验绑定仍 active（解绑/绑定关系被删 → 立即失效）。
// 校验实现与 HTTP 守卫、WS 网关共用同一份（checkCompanionSession）：
// 绑定 active + 打手即会话主体 + 打手未封禁/档案在职 + 老板未封禁（安全审计 F1：不再各写一套）。
if (kind === 'companion') {
const check = await checkCompanionSession(this.dataSource, {
bindingId: session.bindingId,
userId: String(session.userId),
});
if (!check.ok) {
throw new UnauthorizedException(check.message);
}
}
// recharge：每次续期都校验仍在白名单 + 未封禁 + 仍是 admin（移出白名单 → 续期即失效）
if (kind === 'recharge') {
const check = await checkRechargeSession(this.dataSource, { userId: String(session.userId) });
if (!check.ok) {
throw new UnauthorizedException(check.message);
}
}
// 轮换：条件吊销旧 refresh（防竞态：若此刻已被新设备顶掉则拒绝，不误伤新会话）
const r = await this.authTokenRepo.update({ id: session.id, revoked: 0 }, { revoked: 1 });
if ((r.affected ?? 0) !== 1) {
const cur = await this.authTokenRepo.findOne({ where: { jti: payload.jti } });
if (cur?.revoked === 2) {
throw new UnauthorizedException(
kind === 'primary'
? '账号已在其他设备登录'
: kind === 'companion'
? '伴随身份已失效（该打手账号已在别处登录），请重新绑定'
: '受限管理身份已失效（该管理员账号已在别处登录），请重新绑定',
);
}
throw new UnauthorizedException('登录已失效，请重新登录');
}
// 同设备轮换：旧会话按普通失效处理（不触发"顶掉"语义）
// 受限会话沿用自身语义与绑定；primary 调用与历史完全一致（不传 opts，逐字节不变）
return kind === 'primary'
? this.issueTokens(user, false)
: this.issueTokens(user, false, { kind, bindingId: session.bindingId });
}
/** 修改密码（账号与安全，负责人设置需求）：校验旧密码 → 更新 → 吊销全部会话（含当前，需重新登录） */
===== 源文件：server/src/modules/auth/auth.service.ts （第 595-667 行）=====
----- 片段说明：认证服务：登出与会话种类解析 -----
*/
async logout(accessJti: string, refreshToken?: string, userId?: string) {
const kind = await this.resolveSessionKind(accessJti);
// 前置条件①：吊销列表一定包含本次 access 行（即使它已被标记过——重复登出幂等）
```

```
const jtis = new Set<string>([accessJti]);
// 前置条件②：调用方带来的 refreshToken 也算一份（且做归属校验，见下）
if (refreshToken) {
const payload = await this.parseRefreshToken(refreshToken);
if (payload?.jti) jtis.add(String(payload.jti));
}
let revoked = 0;
if (userId) {
// 读"该账号该族的全部活行"（access + refresh）——先查后改，为的是拿到 jti 去踢 WS
const rows: Array<{ jti: string }> = await this.dataSource.query(
SELECT jti FROM auth_token WHERE user_id = ? AND kind = ? AND revoked = 0,
[String(userId), kind],
);
for (const r of rows) jtis.add(String(r.jti));
const res = await this.authTokenRepo.update(
{ userId: String(userId), kind, revoked: 0 },
{ revoked: 1 },
);
revoked = res.affected ?? 0;
} else {
// 理论不可达（控制器必传 userId：守卫已从库中取出登记主体）。留兜底是为了"宁少杀不漏杀"：
// 拿不到主体时只吊销明确点名的那几个 jti，绝不按 user_id 猜（那就可能误伤同账号其他会话）。
const r = await this.authTokenRepo.update({ jti: In([...jtis]), revoked: 0 }, { revoked: 1 });
revoked = r.affected ?? 0;
}
// 调用方带来的 refresh 做一次归属校验 + 兜底吊销：
// 只在"该行确实存在、确实属于本次登出的账号、且 kind 相同"时命中（防"拿别人的 refresh 打我的登出"把它踢下线）。
// 正常路径下它已在上面的同族吊销里被置 1，这里 affected=0，纯兜底。
if (refreshToken && userId) {
try {
const payload = await this.parseRefreshToken(refreshToken);
if (payload?.jti) {
const r = await this.authTokenRepo.update(
{ jti: String(payload.jti), userId: String(userId), kind, revoked: 0 },
{ revoked: 1 },
);
revoked += r.affected ?? 0;
}
} catch {
// refresh 无效则忽略（access 与同族行都已吊销）
}
}
this.logger.log(登出吊销会话 kind=${kind} 行数=${revoked} jti=${[...jtis].length} 个);
return { ok: true, revoked, revokedJtis: [...jtis] };
}
/**
```

- 取 auth_token 中该 jti 的**会话种类**（kind）与主体，作为登出收口的唯一依据。

*

- 为什么必须查库、而不是读令牌载荷里的 kind：载荷是签名者写的，**库中登记行才是权威**
- （与 JwtAuthGuard / refresh() / RealtimeGateway.assertSessionActive 同一条口径：

- "会话种类以库中登记为准，不信任令牌载荷，防自签提权"）。

*

- 为什么取 access 行的 user_id 而不是 binding_id（companion 会话）： companion 行的
- user_id 就是**打手**账号（安全审计 F1-c 已明确），而 companion 的 refresh 行与之同账号同 kind，
- 按 user_id + kind 收口天然覆盖；若按 binding_id 收口反而会牵进 primary（那是明确的越权面）。

*

- 找不到 kind 时回落 primary：登出**只可能**缩到"该账号的 primary 族"，
- 不影响 companion/recharge —— 与全站"未知值一律回落 primary"的既有归一化口径一致。

```
*/
private async resolveSessionKind(accessJti: string): Promise<AuthTokenKind> {
const rows: Array<{ kind: string | null }> = await this.dataSource.query(
SELECT kind FROM auth_token WHERE jti = ? LIMIT 1,
[accessJti],
);
return normalizeKind(rows[0]?.kind);
}
/** 校验并解出 refresh 令牌载荷的 jti（签名/类型/主体不合格一律返回 null，登出流程不因此报错） */
===== 源文件：server/src/modules/auth/auth.service.ts （第 680-760 行）=====
----- 片段说明：认证服务：当前主体（按角色附带档案） -----
/** 当前主体信息（按 role 附带工作端/管理员档案，docs/04） */
async me(authUser: { userId: string; role: string }) {
const user = await this.userRepo.findOne({ where: { id: authUser.userId } });
if (!user) {
throw new UnauthorizedException('账号不存在');
}
const base = {
id: user.id,
username: user.username,
uid: user.uid,
nickname: user.nickname,
avatarMediaId: user.avatarMediaId,
backgroundMediaId: user.backgroundMediaId,
role: user.role,
status: user.status,
level: Number(user.level),
totalSpent: Number(user.totalSpent),
};
if (user.role === 'worker') {
const worker = await this.workerRepo.findOne({ where: { userId: user.id } });
// 分部接单权限（负责人 2026-10-04）：工作端「我的」页展示自己有哪些分部的接单权限。
// 实时查库（无缓存）——管理端刚授/刚撤的权，下次取 me 立刻反映。
const divisionPermissions = worker ? (await this.divisionPerm.listWithFlags(worker.id)) : [];
return {
...base,
worker: worker
? {
workerNo: worker.workerNo,
canTakeOrder: worker.canTakeOrder === 1,
canServeCustomer: worker.canServeCustomer === 1,
status: worker.status,
// 全部分部（含未授权的），客户端只展示已授权的那几个
divisionPermissions,
```

```
}
: null,
};
}
if (user.role === 'admin') {
const admin = await this.adminRepo.findOne({ where: { userId: user.id } });
const role = admin ? await this.adminRoleRepo.findOne({ where: { id: admin.roleId } }) : null;
const permissions: { code: string }[] = role
? await this.dataSource.query(
`SELECT p.code FROM admin_role_permission arp
JOIN permission p ON p.id = arp.permission_id
WHERE arp.role_id = ?`,
[role.id],
)
: [];
return {
...base,
admin: {
roleCode: role?.code ?? null,
permissions: permissions.map((p) => p.code),
// 所有者标记：前端据此显示「封号/注销其他管理员」按钮（服务端仍二次校验）
isOwner: admin?.isOwner === 1,
},
};
}
return base;
}
/**
```

- 签发新令牌对。
- @param replaceOthers 新设备登录=true：顶掉该账号同族旧会话（revoked=2，旧端提示"账号已在其他设备登录"）；
- refresh 轮换=false：旧会话按普通失效处理（revoked=1）
- @param opts.kind 会话种类（默认 primary）——**三类会话互不干扰**（方案甲 + 充值管理员态）：
- - `primary`（普通登录：老板端/工作端/管理端）：只吊销**同账号的其它 primary**，
- `replaceOthers=true` 时置 `revoked=2`（"账号已在其他设备登录"），否则置 1。
- **单设备铁律逐字不变**：同账号第二次登录仍然顶掉第一次（测试钉死）。
- - `companion`（伴随会话）：只吊销同账号的其它 companion（revoked=1），**绝不触碰 primary。**
- - `recharge`（受限充值管理员会话）：只吊销同账号的其它 recharge（revoked=1），**绝不触碰 primary**
- —— 负责人定案"管理员态绝不顶掉 primary"：管理员在管理端的登录必须照常可用。

*

- 为什么 primary 的吊销语句要带 `kind='primary'`（2026-10-06 关键设计取舍）：
- 设计稿 §3.2 承诺「阶段 2 用 companion 会话后两端**不再互相影响**」，R2 把不变式写成
- 「**primary 之间仍互斥**；companion 每设备最多 1 条」。若 primary 登录把 companion 一起顶掉，
- 打手手机上仍在运行的旧版工作端 App **每 2 小时续期一次**就会把老板的打手态踢下线
- （提示"伴随身份已失效"，还要重新输打手账密）——"伴随"就不成立了。
- 加 `kind='primary'` 后：**在只存在 primary 的世界里，语句与历史逐字节等价**
- （没有 companion 行时过滤条件恒真），单设备语义一点没松；
- 而 companion 侧仍有 12h TTL + 每请求校验绑定 + 解绑即吊销三重约束。

```
===== 源文件：server/src/modules/auth/auth.service.ts （第 762-895 行）=====
----- 片段说明：认证服务：令牌签发（primary / companion / recharge 三态） -----
private async issueTokens(user: User, replaceOthers = true, opts?: IssueTokenOptions): Promise<TokenPair> {
const kind: AuthTokenKind = normalizeKind(opts?.kind);
// bindingId 只对 companion 有意义（recharge 的主体就是管理员账号本身，不需要绑定行）
```

```
const bindingId = kind === 'companion' ? (opts?.bindingId ?? null) : null;
if (kind === 'primary') {
// 单设备登录铁律的落地点：同账号再登录 → 旧 primary 置 2（顶号），文案与历史完全一致
await this.authTokenRepo.update(
{ userId: user.id, kind: 'primary', revoked: 0 },
{ revoked: replaceOthers ? 2 : 1 },
);
} else {
/**
```

- 受限会话（companion / recharge）：**只顶掉"同一账号的同类会话"**，primary 行不在命中范围。

*

- 两个 kind 用同一条语句是刻意的 —— 它们的语义完全一致（"非 primary 的受限会话各自成族"），
- 将来再加第三种受限会话也不必改这里。`WHERE kind = ?` 而不是写死 kind，正是为此。

```
*/
await this.authTokenRepo.update({ userId: user.id, kind, revoked: 0 }, { revoked: 1 });
}
// 受限会话统一 12h 硬上限（access 与 refresh 同 TTL）：到点必须重新输账密切换
const restricted = kind !== 'primary';
const accessTtl = restricted ? COMPANION_ACCESS_TTL_SECONDS : ACCESS_TTL_SECONDS;
const refreshTtl = restricted ? COMPANION_REFRESH_TTL_SECONDS : REFRESH_TTL_SECONDS;
const accessJti = randomUUID();
const refreshJti = randomUUID();
const accessExpireAt = new Date(Date.now() + accessTtl * 1000);
const refreshExpireAt = new Date(Date.now() + refreshTtl * 1000);
const accessSecret = this.config.get<string>('JWT_ACCESS_SECRET');
const refreshSecret = this.config.get<string>('JWT_REFRESH_SECRET');
const [accessToken, refreshToken] = await Promise.all([
this.jwt.signAsync(
// kind/bindingId 仅供观测与客户端分流：任何校验都以库中 auth_token 行为准，不信任载荷
{ type: 'access', role: user.role, kind, bindingId },
{ secret: accessSecret, subject: String(user.id), jwtid: accessJti, expiresIn: accessTtl },
),
this.jwt.signAsync(
{ type: 'refresh', role: user.role, kind, bindingId },
{ secret: refreshSecret, subject: String(user.id), jwtid: refreshJti, expiresIn: refreshTtl },
),
]);
await this.authTokenRepo.save([
this.authTokenRepo.create({
jti: accessJti,
userId: user.id,
role: user.role,
kind,
bindingId,
expireAt: accessExpireAt,
}),
this.authTokenRepo.create({
jti: refreshJti,
userId: user.id,
role: user.role,
```

kind,
bindingId,
expireAt: refreshExpireAt,
}),
]);
return { accessToken, refreshToken, expiresIn: accessTtl };
}
/**

- 签发**伴随会话**（companion）—— 只由"老板绑定打手账号"这一条路径调用（`/user/identity/bind`）。

*

- 安全属性（设计稿 R2 的缓解措施，逐条可验证）：
- - 只能由绑定关系签发：`bindingId` 指向 `worker_identity_binding` 的 active 行；
- - 不顶 primary：见 [issueTokens] 的 companion 分支；
- - TTL 12h（access 与 refresh 同）：到点必须用打手账密重新切换；
- - 每请求校验绑定有效性：见 `JwtAuthGuard`（解绑后即使令牌未过期也立即 401）；
- - 可撤销：**任何释放路径**（老板主动解绑 / 打手停用 / 封禁 / 删除）都会按 `binding_id`
- 批量吊销该绑定的全部 companion 会话 —— 实现只有一处：`worker-binding-release.ts` 的
- `releaseBindings()`（安全审计 F1-b 收口；本类不再提供第二份吊销助手，避免将来有人只调它、
- 漏掉"释放绑定 + 审计 + 断开 WS"）。

*/
async issueCompanionTokens(user: User, bindingId: string): Promise<TokenPair> {
return this.issueTokens(user, false, { kind: 'companion', bindingId: String(bindingId) });
}
/**

- 签发**受限充值管理员会话**（`kind='recharge'`）—— 只由
- `POST /user/identity/bind-recharge-admin` 这一条路径调用。

*

- 安全属性（与 companion 同构，逐条可验证）：
- - `recharge_admin` **白名单**是前置条件（不在名单里根本签发不出来）；
- - **不顶 primary**：只吊销该账号的其它 recharge（管理员在管理端的登录照常可用）；
- - TTL 12h（access 与 refresh 同）：到点必须重新输管理员账密；
- - 每请求校验仍在白名单 + 未封禁 + 仍是 admin（见 `JwtAuthGuard` → `checkRechargeSession`）；
- - 可撤销：移出白名单时按 `user_id + kind='recharge'` 批量吊销并断开 WS
- （`RechargeService.removeAdmin()`，唯一入口）。

*/
async issueRechargeTokens(user: User): Promise<TokenPair> {
return this.issueTokens(user, false, { kind: 'recharge' });
}
/**

- 批量吊销某账号的**全部受限充值管理员会话**（`kind='recharge'` 且未吊销），返回被吊销的 jti 列表。

*

- 为什么返回 jti 而不是只回 affected：调用方（`RechargeService.removeAdmin`）需要拿这批 jti
- 去 `RealtimeGateway.disconnectByJti()` **主动断开**已建立的 WS 连接 —— 与 companion 的

- `worker-binding-release.ts` 同一口径（只 UPDATE 不踢线 = 旧连接继续收消息，正是安全审计 F1）。
- **绝不动 primary**：`WHERE kind='recharge'` 把 primary/companion 行排除在外。

```
*/
async revokeRechargeTokens(userId: string): Promise<{ jtis: string[]; affected: number }> {
const rows: Array<{ jti: string }> = await this.dataSource.query(
SELECT jti FROM auth_token WHERE user_id = ? AND kind = 'recharge' AND revoked = 0,
[String(userId)],
--- 第 29 页 ---
```

软件名称：祁连山电竞陪玩服务平台软件 V1.0
);
const r = await this.authTokenRepo.update({ userId: String(userId), kind: 'recharge', revoked: 0 }, { revoked: 1 });
return { jtis: rows.map((x) => String(x.jti)), affected: r.affected ?? 0 };
}
/** 对外展示 ID：U + 10 位数字，唯一冲突重试 */
private async generateUid(): Promise<string> {
for (let i = 0; i < 5; i++) {
const uid = U${randomInt(1_000_000_000, 10_000_000_000)};
const exists = await this.userRepo.findOne({ where: { uid } });
if (!exists) {
return uid;
}
}
throw new BadRequestException('ID 生成失败，请重试');
}
/**

- 邮箱归一化：去首尾空白 + 转小写。
- 为什么必须做：A@qq.com 与 a@qq.com 在 MySQL utf8mb4_0900_ai_ci 下本就判等（唯一索引生效），
- 但 Redis 验证码键是**大小写敏感**的字节比较 —— 不归一化就会出现「用大写取码、小写验码」拿不到码的诡异失败。

*/
private normalizeEmail(raw?: string | null): string | null {
const v = String(raw ?? '').trim().toLowerCase();
return v.length > 0 ? v : null;
}
===== 源文件：server/src/modules/auth/companion-session.check.ts （第 1-27 行，节选）=====
----- 片段说明：伴随会话每请求校验：绑定有效、打手在职、双方未封禁 -----
/**

- 伴随会话（companion）可用性校验 —— **HTTP 守卫与 WebSocket 网关的唯一实现**（安全审计 F1 修复）。

*

- 为什么必须抽成一份（审计 F1 的根因）：
- HTTP 侧 JwtAuthGuard.assertCcompanionUsable 每次请求都校验"绑定 active + 两侧账号状态"，
- 但 RealtimeGateway.assertSessionActive 只查 auth_token(jti/user_id/revoked/expire_at)——
- 于是"绑定已被释放/老板已被封禁"时 **HTTP 已 401，而 WS 连接还能 join 房间并继续收到 msg:new 全文。**
- 把 SQL 复制一份到网关只是把同一个分叉往后推（下一次改校验条件时必然只改一边），
- 所以这里给出**唯一**实现：守卫（每请求）与网关（握手 + 心跳）都调用它。

*

- 校验内容（任一不满足即不可用）：
- 1. binding_id 非空，且绑定行存在、status='active'（解绑 / 封禁 / 停用 / 删除触发的释放都会置 unbound）；
- 2. 绑定里的 worker_user_id **必须等于会话主体**（防绑定 id 被张冠李戴）；
- 3. 打手账号未封禁（user.status=1）、打手档案在职（worker.status=1）；
- 4. **老板账号未封禁且仍存在**（boss_user_id → user.status=1，双保险：专兜历史悬空 active 行）。

*

- 只读、无缓存（每次调用一条 SQL）——"解绑即刻失效"必须建立在实时查库上。

*/
export interface SqlRunner {
query: (sql: string, parameters?: unknown[]) => Promise<unknown>;
}
/** companion 会话失效时的统一中文提示（401：客户端据此清打手态并回到老板态） */

export const COMPANION_GONE = '伴随身份已失效，请重新绑定打手账号';

/ **companion 的**老板账号**侧失效（被封禁/已注销）时的中文提示（401，可操作：重新登录老板账号） */

export const COMPANION_BOSS_GONE = '老板账号已被封禁，伴随身份已失效，请重新登录老板账号';

/** 打手账号被封禁（401；与"解绑"区分，便于客服定位） */

省略说明

本文档按登记要求只提交源程序的**前 30 页与后 30 页**，中部内容不在本文档内。被省略的模块见上文「未纳入本文档的模块」一览表，均为本软件真实存在、可现场演示的模块；如需补充任何模块（含数据库脚本、管理端接口、客户端代码），可按同一抽取口径补交，每页仍为 50 行。

省略不影响审查人员对本软件独创性与功能完整性的判断：前 30 页展示了程序入口、统一请求处理链、认证授权与身份绑定；后 30 页展示了积分账务、订单状态机与结算退款、会话与客服、充值人工审核与金额分账。

第二部分 后 30 页（程序尾部）

软件名称：祁连山电竞陪玩服务平台软件 **V1.0**
===== 源文件：server/src/modules/wallet/wallet.service.ts （第 1-291 行）=====
----- 片段说明：积分钱包：管理员发放 / 回收、等、余快照、工作端收益 -----

```
import { BadRequestException, Injectable, NotFoundException } from '@nestjs/common';
import { InjectDataSource, InjectRepository } from '@nestjs/typeorm';
import { randomBytes, randomInt } from 'crypto';
import { DataSource, EntityManager, QueryFailedError, Repository } from 'typeorm';
import { User } from '../user/user.entity';
import { Wallet } from './wallet.entity';
import { WalletTransaction } from './wallet-transaction.entity';
import { mapMoney } from '../../common/money.transformer';
export interface WalletOpMeta {
ip: string;
requestId: string;
}
/**
```

- 积分服务（docs/06 铁律）：
- - 发放/回收仅 admin:wallet:grant（默认仅超管），客户端金额/余额不可信
- - 事务内：行锁钱包 → 校验 → 更新余额 → 写流水（前后余额快照）
- - 幂等键唯一约束兜底；同键重试返回首次结果，绝不重复入账
- - 回收校验余额充足，禁止负余额

```
*/
@Injectable()
export class WalletService {
constructor(
@InjectRepository(Wallet) private readonly walletRepo: Repository<Wallet>,
@InjectRepository(WalletTransaction) private readonly txnRepo: Repository<WalletTransaction>,
@InjectRepository(User) private readonly userRepo: Repository<User>,
@InjectDataSource() private readonly dataSource: DataSource,
) {}
async grant(adminUserId: string, dto: { userId: string; amount: number; reason?: string; idempotencyKey: string }, meta: WalletOpMeta) {
const target = await this.userRepo.findOne({ where: { id: dto.userId, role: 'user' } });
if (!target) {
throw new NotFoundException('用户不存在');
}
return this.executeChange({
userId: dto.userId,
amount: dto.amount,
changeType: 'grant',
operatorType: 'admin',
operatorId: adminUserId,
source: 'admin-web',
reason: dto.reason ?? null,
idempotencyKey: dto.idempotencyKey,
meta,
});
}
async recover(adminUserId: string, dto: { userId: string; amount: number; reason?: string; idempotencyKey: string }, meta: WalletOpMeta) {
const target = await this.userRepo.findOne({ where: { id: dto.userId, role: 'user' } });
if (!target) {
throw new NotFoundException('用户不存在');
}
return this.executeChange({
```

userId: dto.userId,
amount: -dto.amount,
changeType: 'recover',
operatorType: 'admin',
operatorId: adminUserId,
source: 'admin-web',
reason: dto.reason ?? null,
idempotencyKey: dto.idempotencyKey,
meta,
requireSufficient: true,
});
}
/**

- **事务内**发放积分（充值审核发放专用入口，2026-10-06）。

*

- 为什么要有这个方法，而不是让充值模块自己写 UPDATE：
- 「事务内行锁钱包 → 校验 → 改余额 → 写流水（含前后余额快照）→ 幂等键唯一约束兜底」
- 这一整段是 docs/06 的铁律，**只能有一份实现**。充值发放需要"发积分"与"回写申请状态 +
- 写审计"在**同一个事务**里同生共死（否则会出现"积分发了、申请还是 pending"或反之），
- 所以这里把既有 `executeChange` 开放一个"跑在调用方事务里"的入口 ——
- 逻辑**逐字复用**（本方法只转发，不改任何判定），没有第二条积分路径。

*

- 与 `grant()` 的唯一差别：`executeChange` 用调用方传进来的 `em`（调用方的连接/事务），
- 而不是自己开一个新事务。`grant()` 本身的行为**逐字节未变**（仍走 `dataSource.transaction`）。

*

- @param em 调用方事务的 EntityManager

*/
grantInTx(
em: EntityManager,
adminUserId: string,
dto: { userId: string; amount: number; reason?: string; idempotencyKey: string },
meta: WalletOpMeta,
) {
return this.executeChange(
{
userId: dto.userId,
amount: dto.amount,
changeType: 'grant',
operatorType: 'admin',
operatorId: adminUserId,
source: 'recharge-review',
reason: dto.reason ?? null,
idempotencyKey: dto.idempotencyKey,
meta,
},
em,
);
}
/** 管理端：钱包列表（join 用户信息） */
async adminList(page: number, pageSize: number, keyword: string) {

软件名称：祁连山电竞陪玩服务平台软件 **V1.0**

```
const qb = this.dataSource
.createQueryBuilder(Wallet, 'w')
.innerJoin(User, 'u', 'u.id = w.user_id')
.select(['w.id AS id', 'w.user_id AS userId', 'w.balance AS balance', 'u.username AS username', 'u.uid AS uid', 'u.nickname AS nickname', 'u.role AS role', 'u.level AS level',
'u.total_spent AS totalSpent', 'w.updated_at AS updatedAt']);
if (keyword) {
qb.where('u.username LIKE :kw OR u.uid LIKE :kw OR u.nickname LIKE :kw', { kw: `%${keyword}%` });
}
const total = await qb.clone().select('COUNT(1) AS cnt').getRawOne();
const list = await qb.orderBy('w.updated_at', 'DESC').offset((page - 1) * pageSize).limit(pageSize).getRawMany();
return { list: list.map((r) => mapMoney(r, ['balance', 'totalSpent'])), total: Number(total?.cnt ?? 0), page, pageSize };
}
/** 管理端：积分流水 */
async adminTransactions(page: number, pageSize: number, userId?: string, changeType?: string) {
const qb = this.dataSource
.createQueryBuilder(WalletTransaction, 't')
.innerJoin(User, 'u', 'u.id = t.user_id')
.select(['t.id AS id', 't.transaction_no AS transactionNo', 't.user_id AS userId', 'u.username AS username', 't.change_type AS changeType', 't.amount AS amount',
't.balance_before AS balanceBefore', 't.balance_after AS balanceAfter', 't.operator_type AS operatorType', 't.reason AS reason', 't.created_at AS createdAt']);
if (userId) qb.andWhere('t.user_id = :userId', { userId });
if (changeType) qb.andWhere('t.change_type = :changeType', { changeType });
const total = await qb.clone().select('COUNT(1) AS cnt').getRawOne();
const list = await qb.orderBy('t.id', 'DESC').offset((page - 1) * pageSize).limit(pageSize).getRawMany();
return { list: list.map((r) => mapMoney(r, ['amount', 'balanceBefore', 'balanceAfter'])), total: Number(total?.cnt ?? 0), page, pageSize };
}
/** 用户端：自己的余额（主体 id 取自 JWT，docs/04） */
async userBalance(userId: string) {
const wallet = await this.walletRepo.findOne({ where: { userId } });
return { balance: wallet?.balance ?? 0 };
}
/** 用户端：自己的流水（只读，永不给增积分入口，docs/06） */
async userTransactions(userId: string, page: number, pageSize: number) {
const [list, total] = await this.txnRepo.findAndCount({
where: { userId },
order: { id: 'DESC' },
skip: (page - 1) * pageSize,
take: pageSize,
});
return {
list: list.map((t) => ({
id: t.id,
transactionNo: t.transactionNo,
changeType: t.changeType,
amount: t.amount,
balanceBefore: t.balanceBefore,
balanceAfter: t.balanceAfter,
reason: t.reason,
createdAt: t.createdAt,
})),
total,
page,
pageSize,
```

```
};
}
/**
```

- 核心变更：事务 + 行锁 + 流水快照 + 幂等（docs/06）

```
*
```

- @param em 可选：**调用方的事务**（充值审核发放走这条，见 [grantInTx]）。
- 不传 = 自开事务，行为与历史**逐字节相同**（既有 grant/recover 都不传）。

```
*/
private async executeChange(
input: {
userId: string;
amount: number; // 正=增加（grant），负=减少（recover）
changeType: string;
operatorType: string;
operatorId: string | null;
source: string;
reason: string | null;
idempotencyKey: string;
meta: WalletOpMeta;
requireSufficient?: boolean;
},
em?: EntityManager,
) {
// 幂等先行检查：同键已有流水直接返回首次结果。
// 走调用方事务时必须用它的仓储（em）读，否则读的是另一个连接、看不到本事务未提交的写入。
const repo = em ? em.getRepository(WalletTransaction) : this.txnRepo;
const existing = await repo.findOne({ where: { idempotencyKey: input.idempotencyKey } });
if (existing) {
return this.duplicateResult(existing);
}
const run = async (manager: EntityManager) => {
const wallet = await manager.findOne(Wallet, {
where: { userId: input.userId },
lock: { mode: 'pessimistic_write' },
});
if (!wallet) {
throw new NotFoundException('用户钱包不存在');
}
const before = wallet.balance;
if (input.requireSufficient && before < -input.amount) {
throw new BadRequestException('余额不足');
}
const after = before + input.amount;
await manager.update(Wallet, wallet.id, { balance: after });
const txn = await manager.save(
manager.create(WalletTransaction, {
transactionNo: this.generateTransactionNo(),
userId: input.userId,
changeType: input.changeType,
amount: input.amount,
```

```
balanceBefore: before,
balanceAfter: after,
operatorType: input.operatorType,
operatorId: input.operatorId,
source: input.source,
reason: input.reason,
idempotencyKey: input.idempotencyKey,
ip: input.meta.ip?.slice(0, 64) ?? null,
requestId: input.meta.requestId?.slice(0, 64) ?? null,
}),
);
return { transactionNo: txn.transactionNo, balanceBefore: before, balanceAfter: after, amount: input.amount };
};
// 调用方事务：直接在其中执行（不新开事务 —— 否则"发积分"与调用方的写入无法同生共死）
if (em) return run(em);
try {
return await this.dataSource.transaction(run);
} catch (e) {
// 并发同键：唯一约束兜底，返回首次结果
if (e instanceof QueryFailedError && (e as unknown as { code?: string }).code === 'ER_DUP_ENTRY') {
const dup = await this.txnRepo.findOne({ where: { idempotencyKey: input.idempotencyKey } });
if (dup) {
return this.duplicateResult(dup);
}
}
throw e;
}
}
private duplicateResult(txn: WalletTransaction) {
return {
duplicated: true,
transactionNo: txn.transactionNo,
balanceBefore: txn.balanceBefore,
balanceAfter: txn.balanceAfter,
amount: txn.amount,
};
}
private generateTransactionNo(): string {
const ts = Date.now().toString(36).toUpperCase();
const rand = randomInt(1000, 10000);
const salt = randomBytes(2).toString('hex').toUpperCase();
return T${ts}${salt}${rand};
}
/** 注册时创建钱包（auth 模块事务内调用） */
createWalletInTx(em: EntityManager, userId: string): Promise<Wallet> {
return em.save(em.create(Wallet, { userId, balance: 0 }));
}
/** 工作端我的积分/收益（负责人需求 #6，真实流水聚合）：余额 + 累计结算收入 + 结算次数 + 打赏收益 */
async workerWallet(userId: string) {
const wallet = await this.walletRepo.findOne({ where: { userId } });
```

软件名称：祁连山电竞陪玩服务平台软件 **V1.0**

```
const rows: Array<{ income: string; reversals: string; settlements: string; granted: string; spent: string; tips: string; tipCount: string }> = await this.dataSource.query(
`SELECT
COALESCE(SUM(CASE WHEN change_type = 'order_settlement' THEN amount ELSE 0 END), 0) AS income,
COALESCE(SUM(CASE WHEN change_type = 'settlement_reversal' THEN -amount ELSE 0 END), 0) AS reversals,
SUM(CASE WHEN change_type = 'order_settlement' THEN 1 ELSE 0 END) AS settlements,
COALESCE(SUM(CASE WHEN change_type = 'grant' THEN amount ELSE 0 END), 0) AS granted,
COALESCE(SUM(CASE WHEN amount < 0 THEN -amount ELSE 0 END), 0) AS spent,
-- 打赏收益（负责人 2026-10-04）：独立类型 order_tip_income，明确区分于订单结算
COALESCE(SUM(CASE WHEN change_type = 'order_tip_income' THEN amount ELSE 0 END), 0) AS tips,
SUM(CASE WHEN change_type = 'order_tip_income' THEN 1 ELSE 0 END) AS tipCount
FROM wallet_transaction WHERE user_id = ?`,
[userId],
);
const r = rows[0];
return {
balance: wallet?.balance ?? 0,
// 累计获得积分：订单结算 + 管理员发放（负责人 2026-09-25：工作端完整积分入口）
totalIncome: Number(r?.income ?? 0) - Number(r?.reversals ?? 0),
totalGranted: Number(r?.granted ?? 0),
totalSpent: Number(r?.spent ?? 0),
settlementCount: Number(r?.settlements ?? 0),
// 打赏收益（老板打赏 100% 到手，平台不抽成）
tipIncome: Number(r?.tips ?? 0),
tipCount: Number(r?.tipCount ?? 0),
};
}
}
===== 源文件：server/src/modules/order/order.service.ts （第 94-168 行）=====
----- 片段说明：订单状态机：合法迁移表（唯一依据） -----
const TRANSITIONS: Record<OrderStatus, OrderStatus[]> = {
pending_take: ['taken', 'pending_dispatch', 'canceled'],
pending_dispatch: ['taken', 'pending_take', 'canceled'],
taken: ['in_progress', 'canceled'],
in_progress: ['pending_confirm', 'abnormal', 'canceled'],
pending_confirm: ['completed', 'abnormal'],
completed: ['after_sale'],
abnormal: ['canceled', 'completed', 'after_sale'],
after_sale: ['completed', 'canceled'],
canceled: [],
};
/**
```

- 订单服务（docs/05 + 负责人结算需求）：
- - 下单：服务端查库定价 → 事务内扣款+建单+价格快照+流水；幂等键唯一兜底
- - 结算：老板确认完成 = 事务内【校验归属/状态/打手 → 打手+分成% → 平台+抽成% → 结算记录 → 状态流转 → 日志】
- （分成比例读 system_setting.platform_commission_rate，默认 25% 抽成 = 打手 75%）
- - 一个订单最多结算一次：order_settlement.order_id 唯一约束 + 订单行锁 + 状态检查 三重保证

```
*/
@Injectable()
export class OrderService implements OnApplicationBootstrap {
private readonly logger = new Logger('Order');
private sweepTimer: NodeJS.Timeout | null = null;
constructor(
```

软件名称：祁连山电竞陪玩服务平台软件 **V1.0**
@InjectRepository(Order) private readonly orderRepo: Repository<Order>,
@InjectRepository(OrderSettlement) private readonly settlementRepo: Repository<OrderSettlement>,
@InjectDataSource() private readonly dataSource: DataSource,
private readonly settings: SystemSettingService,
private readonly realtime: RealtimeGateway,
private readonly level: LevelService,
private readonly chat: ChatService,
/ **分部接单权限（负责人 2026-10-04）：大厅/抢单/接单/派单/指定护航员全部走它，**每次实时查库** */
private readonly divisionPerm: WorkerDivisionPermissionService,
/**

- 分部（负责人 2026-10-05）：**防误单**用 —— 建设中分部（瓦端/瓦手）的文案是「敬请期待」，
- 若管理端误挂了商品，这里必须中文拒绝，而不是让老板付了积分才发现没人接单。

*/
private readonly divisions: DivisionService,
) {}
/** 超时扫描（派单确认 + 待接单超时）：启动立即扫一次（覆盖停机窗口）+ 每 30s 周期（docs/05：条件 UPDATE 幂等，禁 Redis 锁双源） */
onApplicationBootstrap(): void {
const tick = () => {
this.sweepExpiredDispatches().catch((e) => this.logger.warn(派单超时扫描失败：${(e as Error).message}));
this.sweepPendingTakeTimeout().catch((e) => this.logger.warn(待接单超时扫描失败：${(e as Error).message}));
};
tick();
this.sweepTimer = setInterval(tick, 30_000);
// 进程退出时清理（非关键路径，Node 退出自然回收）
this.sweepTimer.unref?.();
}
// ---------- 下单 ----------
async createOrder(
userId: string,
dto: {
productId: string;
quantity: number;
remark?: string;
idempotencyKey: string;
gameZone: string;
gamePlayerId: string;
contactPhone?: string;
workerMode?: string;
designatedWorkerId?: string;
},
meta: { ip: string; requestId: string },
) {
// 幂等先行：同键已有订单直接返回（docs/06）
const existing = await this.orderRepo.findOne({ where: { idempotencyKey: dto.idempotencyKey } });
if (existing) {
return this.orderView(existing, true);
}
===== 源文件：server/src/modules/order/order.service.ts （第 468-570 行）=====
----- 片段说明：订单完成结算事务：行锁 + 三重幂等 + 服务端分账 -----
private async completeOrderTx(
em: EntityManager,
where: { id: string; userId?: string },

```
allowedFrom: OrderStatus[],
meta: { ip: string; requestId: string },
actor: { operatorType: 'user' | 'admin' | 'system'; operatorId: string | null; source: 'client' | 'admin' | 'system'; remark: string },
opts: { allowAlreadySettled?: boolean } = {},
): Promise<OrderCompletion> {
const order = await em.findOne(Order, { where, lock: { mode: 'pessimistic_write' } });
if (!order) {
throw new NotFoundException('订单不存在');
}
this.assertStatus(order, allowedFrom, '当前状态不允许完成订单');
if (!order.workerId) {
throw new BadRequestException('订单无接单打手，无法结算');
}
// order.workerId 是 worker 档案 id，需映射 user_id（钱包/会话/WS 通道都用 user_id）
const workerProfile = await em.findOne(Worker, { where: { id: order.workerId } });
if (!workerProfile || workerProfile.status !== 1) {
throw new BadRequestException('打手档案不存在或已停用');
}
// 三重防重复结算：状态检查（上）+ 结算记录唯一约束 + 行锁串行化
const existing = await em.findOne(OrderSettlement, { where: { orderId: order.id } });
if (existing && !opts.allowAlreadySettled) {
throw new BadRequestException('订单已结算，请勿重复操作');
}
const total = order.totalAmount;
let settlement = existing;
// 生效抽成比例：读 system_setting（唯一真相，管理端可改，改完下一单立即生效；不缓存见 service 注释）
const commissionRate = await this.settings.platformCommissionRate();
const split = splitAmount(total, commissionRate);
if (!existing) {
// 服务器分账（按订单快照金额，绝不用商品当前价格；金额支持小数 → 四舍五入到 0.01）
const { workerAmount, platformAmount, workerPercentage, platformPercentage } = split;
// 打手 +workerPercentage%（工作端自己的钱包）
const workerWallet = await em.findOne(Wallet, { where: { userId: workerProfile.userId }, lock: { mode: 'pessimistic_write' } });
if (!workerWallet) {
throw new BadRequestException('打手钱包不存在');
}
await em.update(Wallet, workerWallet.id, { balance: workerWallet.balance + workerAmount });
await this.insertTxn(em, {
userId: workerProfile.userId, changeType: 'order_settlement', amount: workerAmount,
balanceBefore: workerWallet.balance, balanceAfter: workerWallet.balance + workerAmount,
operatorType: actor.operatorType, operatorId: actor.operatorId, source: 'order_settlement',
reason: 订单完成结算 ${workerPercentage}%, orderId: order.id, idempotencyKey: settle:${order.id}, meta,
});
// 平台 +platformPercentage%（平台账户钱包）
const platformUser = await em.findOne(User, { where: { uid: 'PLATFORM1' } });
if (!platformUser) {
throw new BadRequestException('平台账户不存在');
}
const platformWallet = await em.findOne(Wallet, { where: { userId: platformUser.id }, lock: { mode: 'pessimistic_write' } });
if (!platformWallet) {
```

throw new BadRequestException('平台钱包不存在');
}
await em.update(Wallet, platformWallet.id, { balance: platformWallet.balance + platformAmount });
await this.insertTxn(em, {
userId: platformUser.id, changeType: 'platform_commission', amount: platformAmount,
balanceBefore: platformWallet.balance, balanceAfter: platformWallet.balance + platformAmount,
operatorType: 'system', operatorId: null, source: 'order_settlement',
reason: 平台抽成 ${platformPercentage}%, orderId: order.id, idempotencyKey: platform:${order.id}, meta,
});
// 唯一结算记录（数据库级幂等兜底）：比例按当时生效配置落库留痕（历史单据可回溯）
settlement = await em.save(
em.create(OrderSettlement, {
settlementNo: this.generateSettlementNo(),
orderId: order.id,
orderAmount: total,
workerId: order.workerId,
workerAmount,
workerPercentage,
platformAmount,
platformPercentage,
requestId: meta.requestId?.slice(0, 64) ?? null,
}),
);
// 销量累加 + 打手统计（M7，事务内同一数据源；已结算单不重复累加）
await em.query('UPDATE product SET sales_count = sales_count + ? WHERE id = ?', [order.quantity, order.productId]);
await em.query(
`INSERT INTO worker_stats (worker_id, total_orders, completed_orders, total_amount)
VALUES (?, 1, 1, ?)
ON DUPLICATE KEY UPDATE total_orders = total_orders + 1, completed_orders = completed_orders + 1, total_amount = total_amount + ?`,
[order.workerId, total, total],
);
this.logger.log(
订单结算成功 orderNo=${order.orderNo} settlement=${settlement.settlementNo} 抽成=${platformPercentage}% worker=${workerAmount}
platform=${platformAmount},
);
} else {
this.logger.log(订单已完成结算，仅补状态流转 orderNo=${order.orderNo} settlement=${settlement?.settlementNo});
}
// 状态流转 + 状态日志 + 两端实时（both 老板端与工作端订单列表立即变「已完成」）
await em.update(Order, order.id, { status: 'completed' });
await this.logStatus(em, order.id, order.status, 'completed', actor.operatorType, actor.operatorId, actor.source, meta, actor.remark);
this.realtime.emitToUser(order.userId, 'order:update', { orderId: order.id, status: 'completed' });
this.realtime.emitToUser(workerProfile.userId, 'order:update', { orderId: order.id, status: 'completed' });
===== 源文件：server/src/modules/order/order.service.ts （第 1167-1265 行）=====
----- 片段说明：退款事务：冲正结算 + 回退打手（同一事） -----
async adminRefundOrderTx(
em: EntityManager,
order: Order,
fromStatus: OrderStatus | string,
adminUserId: string,
opts: { reason: string; logRemark: string },
meta: { ip: string; requestId: string },
): Promise<DirectOutcome> {

```
const settlement = await em.findOne(OrderSettlement, { where: { orderId: order.id } });
if (settlement) {
await this.reverseSettlementAndRefund(em, order, settlement, adminUserId, opts.reason, meta);
} else {
await this.refundAndClose(em, order, 'admin', opts.reason, meta);
}
await em.update(Order, order.id, { status: 'canceled' });
await this.logStatus(em, order.id, fromStatus, 'canceled', 'admin', adminUserId, 'admin', meta, opts.logRemark);
// 售后退款 = 把订单置 canceled 的路径之一：无其它未完成订单 → 删除会话；还有 → 保持 open 不动
return this.settleCanceledDirectChatTx(em, order);
}
/** 已结算订单退款：按结算记录冲正打手与平台（金额取落库值，与当前配置无关），再全额退用户（打手余额不足则拒绝并提示人工处理） */
private async reverseSettlementAndRefund(
em: EntityManager,
order: Order,
settlement: OrderSettlement,
adminUserId: string,
reason: string,
meta: { ip: string; requestId: string },
) {
// 1. 打手退回该单结算金额（余额校验，防负余额）
const worker = await em.findOne(Worker, { where: { id: settlement.workerId } });
if (!worker) throw new BadRequestException('打手档案不存在，无法自动冲正（请人工处理）');
const workerWallet = await em.findOne(Wallet, { where: { userId: worker.userId }, lock: { mode: 'pessimistic_write' } });
if (!workerWallet || workerWallet.balance < settlement.workerAmount) {
throw new BadRequestException('打手钱包余额不足，无法自动冲正退款（请人工处理）');
}
await em.update(Wallet, workerWallet.id, { balance: workerWallet.balance - settlement.workerAmount });
await this.insertTxn(em, {
userId: worker.userId, changeType: 'settlement_reversal', amount: -settlement.workerAmount,
balanceBefore: workerWallet.balance, balanceAfter: workerWallet.balance - settlement.workerAmount,
operatorType: 'admin', operatorId: adminUserId, source: 'admin',
reason: 售后退款冲正结算（${settlement.settlementNo}）：${reason}, orderId: order.id, idempotencyKey: settlement-reversal:${order.id}:worker, meta,
});
await this.rollbackWorkerStatsTx(em, settlement);
// 2. 平台退回该单抽成金额
const platformUser = await em.findOne(User, { where: { uid: 'PLATFORM1' } });
if (platformUser) {
const platformWallet = await em.findOne(Wallet, { where: { userId: platformUser.id }, lock: { mode: 'pessimistic_write' } });
if (platformWallet) {
await em.update(Wallet, platformWallet.id, { balance: platformWallet.balance - settlement.platformAmount });
await this.insertTxn(em, {
userId: platformUser.id, changeType: 'platform_commission_reversal', amount: -settlement.platformAmount,
balanceBefore: platformWallet.balance, balanceAfter: platformWallet.balance - settlement.platformAmount,
operatorType: 'admin', operatorId: adminUserId, source: 'admin',
reason: 售后退款冲正平台抽成（${settlement.settlementNo}）, orderId: order.id, idempotencyKey: settlement-reversal:${order.id}:platform, meta,
});
}
}
// 3. 用户全额退款
```

软件名称：祁连山电竞陪玩服务平台软件 V1.0
const userWallet = await em.findOne(Wallet, { where: { userId: order.userId }, lock: { mode: 'pessimistic_write' } });
if (!userWallet) throw new BadRequestException('用户钱包不存在');
await em.update(Wallet, userWallet.id, { balance: userWallet.balance + order.totalAmount });
await this.insertTxn(em, {
userId: order.userId, changeType: 'order_refund', amount: order.totalAmount,
balanceBefore: userWallet.balance, balanceAfter: userWallet.balance + order.totalAmount,
operatorType: 'admin', operatorId: adminUserId, source: 'admin',
reason: 售后退款：${reason}, orderId: order.id, idempotencyKey: refund:${order.id}, meta,
});
// 退款回退累计消费（等级不自动降级）
await this.level.refundSpentInTx(em, order.userId, order.totalAmount);
}
/**

- 退款冲正时**同步回退打手战绩**（worker_stats，M7）—— 与钱包冲正**同一事务**，钱与战绩必须一致。

*

- ## 为什么必须有它（真 bug，2026-10-06 修）

*

- 结算路径（本文件 settleTx）在事务内做了
- INSERT INTO worker_stats … ON DUPLICATE KEY UPDATE +1/+1/+amount,
- 但**退款路径原先只写 wallet 冲正流水、从不回退 worker_stats** ——
- 于是「走完全合法的售后退款链」之后，**钱包退干净了、战绩却永久虚高**:
- GET /worker/my-stats 与打手态「我的」继续显示不该有的完成单数与金额。
- 实证（修前）：打手 168 多算 1 单（order 785 已退款）、打手 170 多算 1 单（order 907）。

*

- ## 三条口径（与负责人 2026-10-06 定案一致）

*

- 1. **对称**：结算时加的就是 settlement.orderAmount（落库值），这里就减**同一个落库值**，
- **不重算**——与上面冲正金额「不回读当前配置」同一口径（改配置不会让老单冲正错位）。
- 2. **不许负值**：用**条件 UPDATE**（三项都够减才动），而不是 GREATEST(0, …)。
- 选条件 UPDATE 的理由：GREATEST 会把「行不存在 / 已被别的单减过 / 数据本来就错」
- **静默伪装成「减成功了」**，负值是被夹住了但异常被吞掉；条件 UPDATE 则如实返回
- affectedRows = 0，我们据此打告警，异常看得见。宁可少减一次并告警，也不假装成功。
- 3. **幂等（同一单最多回退一次）**：本方法只在 settlement-reversal:{orderId}:worker
- 这条流水**插入成功之后**才被调用，而该 idempotency_key 有**唯一约束**
- （UQ_wallet_txn_idem）→ 同一张单重复退款会在插流水那一步就抛冲突并整体回滚，
- 本回退天然最多执行一次，无需再引入第二个幂等键。

*

- **失败不阻断退款**：回退失败只告警（钱必须退给老板，这是第一优先），
- 与结算侧「战绩累加失败不阻断结算」的既有取舍对称。

===== 源文件：server/src/modules/chat/chat.service.ts （第 416-541 行）=====
----- 片段说明：会话列表：三类分区与未读汇总 -----
async listUserConversations(userId: string) {
const rows: Array<{
id: string; type: string; user_id: string; peer_user_id: string | null; order_id: string | null; name: string | null;
unread_user: number; unread_worker: number; unread_peer: number; member_unread: number | null; last_seq: string; updated_at: Date;
status: string; closed_at: Date | null; closed_by: string | null;

```
peer_nickname: string | null; peer_uid: string | null; peer_avatar: string | null;
}> = await this.convRepo.manager.query(
`SELECT c.id, c.type, c.user_id, c.peer_user_id, c.order_id, c.name, c.unread_user, c.unread_worker, c.unread_peer, c.last_seq, c.updated_at,
c.status, c.closed_at, c.closed_by,
u.nickname AS peer_nickname, u.uid AS peer_uid, u.avatar_media_id AS peer_avatar,
NULL AS member_unread
--- 第 41 页 ---
```

```
FROM chat_conversation c
LEFT JOIN user u ON u.id = c.peer_user_id
WHERE (c.user_id = ? AND c.type = 'cs')
OR (c.type = 'friend' AND (c.user_id = ? OR c.peer_user_id = ?))
OR (c.type = 'worker' AND (c.user_id = ? OR c.peer_user_id = ?))
ORDER BY c.updated_at DESC`,
[userId, userId, userId, userId, userId],
);
if (!rows.length) return { list: [], total: 0 };
const ids = rows.map((r) => r.id);
const lastRows: Array<{ conversation_id: string; content: string; msg_type: string; sender_type: string; sender_id: string; recalled: number; created_at: Date }> =
await this.convRepo.query(
`SELECT m.conversation_id, m.content, m.msg_type, m.sender_type, m.sender_id, m.recalled, m.created_at
FROM chat_message m
JOIN (SELECT conversation_id, MAX(seq) AS ms FROM chat_message WHERE conversation_id IN (?) GROUP BY conversation_id) t
ON t.conversation_id = m.conversation_id AND t.ms = m.seq`,
[ids],
);
const lastMap = new Map(lastRows.map((r) => [String(r.conversation_id), r]));
return {
list: rows.map((r) => {
const last = lastMap.get(String(r.id));
const isOwner = String(r.user_id) === String(userId);
// cs：属主看 unread_user，客服看 unread_worker；好友/打手直聊：属主看 unread_user，对端看 unread_peer
const unread = r.type === 'cs'
? (isOwner ? Number(r.unread_user) : Number(r.unread_worker))
: (isOwner ? Number(r.unread_user) : Number(r.unread_peer));
return {
id: r.id,
type: r.type,
name: null,
orderId: null,
// 好友与打手直聊：都展示对方资料（打手直聊的对方即接单打手）
peer: (r.type === 'friend' || r.type === 'worker') && r.peer_user_id
? {
userId: String(r.peer_user_id) === String(userId) ? String(r.user_id) : String(r.peer_user_id),
nickname: r.peer_nickname ?? '',
uid: r.peer_uid ?? '',
avatarMediaId: r.peer_avatar,
}
: null,
unread,
lastSeq: Number(r.last_seq),
status: r.status,
closedAt: r.closed_at,
closedBy: r.closed_by,
lastMessage: last
? {
content: last.content,
msgType: last.msg_type,
```

```
senderType: last.sender_type,
senderId: String(last.sender_id),
recalled: Number(last.recalled) === 1,
createdAt: last.created_at,
}
: null,
updatedAt: r.updated_at,
};
}),
total: rows.length,
};
}
/**
```

- 工作端会话列表：**我作为客服被指派的 CS 会话 ∪ 我自己作为发起方的 CS 会话** + 我的打手直聊 + 我的好友会话。

*

- 缺口 4（负责人 2026-10-06 裁决）：打手态「联系客服」走的是 `GET /user/conversation`，它建的是
- **以打手账号为 user 侧**的 CS 会话（`chat_conversation.user_id = 打手账号`，没有指派客服）。
- 原实现只返回「被指派的会话」，于是打手联系了客服却在客服段看不到这条会话、也永远看不到客服回复
- （功能坏掉）。现在集合 = 被指派（`a.id 非空`） ∪ 打手作起方（ `c.user_id = 我`）。

*

- 不泄漏别人的会话（硬约束）：两个条件都是**当事方**判定 —— 要么我是客服侧（有 active 指派），
- 要么我是发起方（会话属主）。与该账号无关的第三条会话永远不会进这个列表（测试钉死）。

```
*/
async workerConversations(workerUserId: string) {
const hasCs = await this.assignRepo.manager
.findOne(Worker, { where: { userId: workerUserId, status: 1 } })
.then((w) => !!w && w.canServeCustomer === 1)
.catch(() => false);
const rows: Array<{
id: string; user_id: string; nickname: string; type: string;
unread_user: number; unread_worker: number; unread_peer: number; last_seq: string; status: string; updated_at: Date;
}> = await this.convRepo.manager.query(
`SELECT c.id, c.user_id, u.nickname, c.type, c.unread_user, c.unread_worker, c.unread_peer, c.last_seq, c.status, c.updated_at
FROM chat_conversation c
LEFT JOIN user u ON u.id = c.user_id
LEFT JOIN chat_conversation_assign a ON a.conversation_id = c.id AND a.status = 'active' AND a.worker_user_id = ?
WHERE (c.type = 'cs' AND (a.id IS NOT NULL OR c.user_id = ?))
OR (c.type = 'worker' AND c.peer_user_id = ?)
OR (c.type = 'friend' AND (c.user_id = ? OR c.peer_user_id = ?))
ORDER BY c.updated_at DESC`,
[workerUserId, workerUserId, workerUserId, workerUserId, workerUserId],
);
return {
hasCsPermission: hasCs,
list: rows.map((r) => ({
id: r.id,
type: r.type,
name: null,
orderId: null,
// CS 会话：我是发起方时昵称就是我自己的账号昵称（客户端客服段固定显示「祁连山客服」，不看这个字段）
```

```
userNickname: r.nickname ?? '用户',
// CS 会话未读分两侧（缺口 4 新增的"自发起"行必须按发起方口径取）：
// - 我是发起方（user_id = 我）→ unread_user（客服回复我的未读）；
// - 我是被指派的客服 → unread_worker（老板消息的客服侧未读）。
// 打手直聊：属主是老板、工作端在对端位置 → unread_peer（既有口径逐字未变）。
unread: r.type === 'cs'
? (String(r.user_id) === String(workerUserId) ? Number(r.unread_user) : Number(r.unread_worker))
: Number(r.unread_peer),
lastSeq: Number(r.last_seq),
status: r.status,
updatedAt: r.updated_at,
})),
};
}
```

```
===== 源文件：server/src/modules/chat/chat.service.ts （第 706-920 行）=====
----- 片段说明：客服大厅：待接入池、抢占接入、释放回池 -----
async csHall(workerUserId: string, page: number, pageSize: number) {
await this.workerOf(workerUserId);
// 总数与列表同源：同一个 CS_POOL_SQL 内插在两处（这条 COUNT 与下面列表的 WHERE），
// 下面返回的 waitingTotal 又直接取 total 这同一个变量 → 三者不可能口径漂移。
const totalRows: Array<{ cnt: number | string }> = await this.convRepo.manager.query(
SELECT COUNT(*) AS cnt FROM chat_conversation c WHERE ${CS_POOL_SQL},
);
const total = Number(totalRows?.[0]?.cnt ?? 0);
const rows: Array<{
id: string; created_at: Date; first_at: Date | null; first_type: string | null; first_content: string | null;
first_recalled: number | null; msg_count: number | string; nickname: string | null; user_uid: string | null;
waited_seconds: number | string;
}> = await this.convRepo.manager.query(
`SELECT c.id, c.created_at, u.nickname, u.uid AS user_uid,
fm.msg_type AS first_type, fm.content AS first_content, fm.recalled AS first_recalled,
fm.created_at AS first_at,
(SELECT COUNT(*) FROM chat_message m WHERE m.conversation_id = c.id) AS msg_count,
TIMESTAMPDIFF(SECOND, ${CS_WAITING_SINCE_SQL}, NOW()) AS waited_seconds
FROM chat_conversation c
LEFT JOIN user u ON u.id = c.user_id
LEFT JOIN chat_message fm ON fm.id = (
SELECT m2.id FROM chat_message m2
WHERE m2.conversation_id = c.id AND m2.sender_type = 'user'
ORDER BY m2.seq ASC LIMIT 1)
WHERE ${CS_POOL_SQL}
ORDER BY ${CS_WAITING_SINCE_SQL} ASC, c.id ASC
LIMIT ? OFFSET ?`,
[pageSize, (page - 1) * pageSize],
);
const onlineCsUserIds = await this.onlineCsWorkerIds();
return {
list: rows.map((r) => {
const waitedSeconds = Math.max(0, Number(r.waited_seconds ?? 0));
return {
conversationId: String(r.id),
// 发起方（老板；也可能是打手态账号「联系客服」发起的会话）昵称 + 中文 ID
```

```
userNickname: r.nickname ?? '用户',
userUid: r.user_uid ?? '',
preview: this.csPreviewOf(r.first_type, r.first_content, Number(r.first_recalled) === 1),
msgCount: Number(r.msg_count ?? 0),
waitedSeconds,
waitedText: this.humanWait(waitedSeconds),
// 等待起点（= 排序依据）+ 会话创建时刻，客户端可原样展示。
// 新口径下 first_at 必然存在（进池就要求有老板消息），?? r.created_at 只是防御兜底
waitingSince: r.first_at ?? r.created_at,
createdAt: r.created_at,
};
}),
total,
page,
pageSize,
/** 与 total 同值：显式给一个"待接入总数"的名字，客户端顶部可直接显示"当前 N 人等待" */
waitingTotal: total,
/** 当前有客服权限且在线的打手数（口径见 [onlineCsWorkerIds]） */
onlineCsCount: onlineCsUserIds.length,
};
}
/**
```

- 主动接入（`POST /worker/cs/hall/:conversationId/claim`）—— **抢占式，谁先点谁得。**

*

- ## 并发安全（项目铁律：条件写判 affectedRows，禁 Redis 锁双源）
- 1. **客服权限实时查库**（`workerOf`，无缓存）→ 403 中文；
- 2. 事务内 `SELECT ... FOR UPDATE` **锁会话行**（与 `sendMessage` 抢的是同一张表的同一行）
- → 所有 claim 在这里串行化，第二个请求必然在第一个提交**之后**才读到 active 行；
- 3. 锁内**复核**「仍是 `cs` + `open` + 无 active 指派」——复核结果就是最终判定；
- 4. **条件写 + affectedRows 判结果**：`INSERT ... status='active'` 的 `affectedRows` 必须是 1，
- 否则（0 行）一律 409 中文，**绝不静默成功。**数据来源只有这张表，没有第二把锁、没有 Redis。
- 5. **数据库层兜底**：唯一键 `UK_cca_active`（生成列 `active_conversation_id`）保证"同一会话最多一行 active"，
- 即使将来有人绕过行锁（管理端改派、运维 SQL、新写的接口）也插不进第二行 →
- 捕获 `ER_DUP_ENTRY` 后同样回**中文 409**（而不是把 SQL 错误原样抛成 500）。

*

- ## 幂等
- 自己已经接过（active 行就是自己）→ 直接返回 `alreadyClaimed: true`，不报错、不产生第二行
- —— 客户端重复点/网络重发不会看到红字。

*

- ## 存在性口径（安全审计 F3 的既有教训）
- 「会话不存在」与「存在但不是客服会话」一律 **404「会话不存在」**（不向客服泄漏平台上还有哪些
- 别的类型的会话）；「已结束」→ 400 中文。管理端 assign 的既有文案与状态码**未改。**

*

- ## 留痕
- 真正写入的那一次由控制器写 `operation_log`（`module='chat'`、`action='chat:claim'`，
- 与 `admin/chat` 的 `chat:assign` **同一张表、同一套字段**）—— 一个会话的接入史

- （管理员改派 + 客服自主接入 + 退回池子）永远能在同一处回看。

*/
async claimCsConversation(workerUserId: string, conversationId: string) {
const worker = await this.workerOf(workerUserId);

```
const outcome = await this.convRepo.manager.transaction(async (em) => {
// ② 行锁：SELECT ... FOR UPDATE（事务内；与 sendMessage 同一把锁 → 并发 claim 串行化）
const locked = await em.findOne(ChatConversation, {
where: { id: conversationId },
lock: { mode: 'pessimistic_write' },
});
if (!locked || locked.type !== 'cs') throw new NotFoundException('会话不存在');
if (locked.status !== 'open') throw new BadRequestException('该咨询已结束，无法接入');
// ③ 锁内复核：此刻读到的 active 行不可能再被别的 claim 插入（插入方必须先拿同一把行锁）
const active = await em.find(ChatConversationAssign, { where: { conversationId, status: 'active' } });
const mine = active.find((a) => String(a.workerUserId) === String(workerUserId));
if (mine) return { alreadyClaimed: true };
if (active.length > 0) throw new ConflictException('该咨询已被其他客服接入');
// ④ 条件写：affectedRows 是"接入成功"的唯一判据
let affected = 0;
try {
const res: { affectedRows?: number } = await em.query(
`INSERT INTO chat_conversation_assign (conversation_id, worker_user_id, status, assigned_by)
VALUES (?, ?, 'active', ?)`,
[conversationId, workerUserId, workerUserId],
);
affected = Number(res?.affectedRows ?? 0);
} catch (e) {
// 唯一键 UK_cca_active：有别的写入路径先插了 active 行（绕过行锁）→ 同样是"已被接入"
if ((e as { driverError?: { code?: string } }).driverError?.code === 'ER_DUP_ENTRY') {
throw new ConflictException('该咨询已被其他客服接入');
}
throw e;
}
if (affected !== 1) throw new ConflictException('该咨询已被其他客服接入');
return { alreadyClaimed: false };
});
const nickname = (await this.nicknamesOf([workerUserId])).get(String(workerUserId)) ?? '';
const payload = {
conversationId: String(conversationId),
workerUserId: String(workerUserId),
workerNickname: nickname,
workerNo: worker.workerNo,
};
// ① 抢到的人：沿用既有事件名 chat:assigned（打手态消息页已监听 → 自动刷新会话列表）
this.realtime.emitToUser(workerUserId, 'chat:assigned', payload);
if (!outcome.alreadyClaimed) {
// ② 大厅其他人：该条从列表移除。载荷字段名按需求口径 {conversationId, byNickname} 命名
//    （by* = "被谁接走了"，与 ①/③ 的 worker* = "接他的人是谁" 区分：一个给大厅，一个给当事人）
void this.emitToOnlineCs(
'cs:claimed',
{
conversationId: String(conversationId),
byUserId: String(workerUserId),
byNickname: nickname,
```

byWorkerNo: worker.workerNo,
},
workerUserId,
);
// ③ 老板侧：同一个 chat:assigned 事件（载荷含客服昵称/工号，老板端据此显示「客服 X 已接入」）
const conv = await this.convRepo.findOne({ where: { id: conversationId } });
if (conv) this.realtime.emitToUser(conv.userId, 'chat:assigned', payload);
}
return {
ok: true,
conversationId: String(conversationId),
alreadyClaimed: outcome.alreadyClaimed,
workerUserId: String(workerUserId),
workerNo: worker.workerNo,
};
}
/**

- 退回池子（`POST /worker/cs/hall/:conversationId/release`）—— **负责人尚未定案，由开关控制开放。**

*

- 语义：把自己的那行 active 指派置为 `released`（**历史保留**，不删除）→ 会话重新进入待接入池
- → 大厅里其他人重新看到它。

*

- 判定同样是**条件写判 affectedRows**（`WHERE ... AND worker_user_id = ? AND status='active'`）：
- - 改到 1 行 = 退回成功（通知三方）；
- - 0 行分两种：我已经退回过（历史行 `released` 存在）→ **幂等成功**；
- 其余（没接过 / 是别人在接 / 会话已结束）→ 中文 400。

*

- 开关：`system_setting.cs_hall_release_enabled`（默认 1=开放，`{"value":0}`=关闭 → 403 中文）。
- 读侧实时读库、无缓存 → 关掉下一次请求立刻生效。要彻底删除见 `cs-hall.ts` 顶部说明。

*/
async releaseCsConversation(workerUserId: string, conversationId: string) {
await this.workerOf(workerUserId);
const enabled = await this.settings.value(CS_HALL_RELEASE_KEY, CS_HALL_RELEASE_DEFAULT);
if (enabled === CS_HALL_RELEASE_DISABLED_VALUE) {
throw new ForbiddenException('客服退回功能未开放');
}
const conv = await this.convRepo.findOne({ where: { id: conversationId } });
if (!conv || conv.type !== 'cs') throw new NotFoundException('会话不存在');
if (conv.status !== 'open') throw new BadRequestException('该咨询已结束，无需退回');
const res: { affectedRows?: number } = await this.convRepo.manager.query(
`UPDATE chat_conversation_assign SET status = 'released'
WHERE conversation_id = ? AND worker_user_id = ? AND status = 'active'`,
[conversationId, workerUserId],
);
if (Number(res?.affectedRows ?? 0) !== 1) {
const released = await this.assignRepo.findOne({
where: { conversationId, workerUserId: String(workerUserId), status: 'released' },
});
if (released) return { ok: true, conversationId: String(conversationId), alreadyReleased: true };
throw new BadRequestException('你不是该咨询的接入客服，无法退回');

```
}
// 通知三方：本人（沿用 chat:unassigned，打手态已监听）、老板、大厅其他人（cs:new → 重新出现）
this.realtime.emitToUser(workerUserId, 'chat:unassigned', { conversationId: String(conversationId), by: 'worker' });
this.realtime.emitToUser(conv.userId, 'chat:unassigned', { conversationId: String(conversationId), by: 'worker' });
void this.announceCsNew(conversationId, 'released');
return { ok: true, conversationId: String(conversationId), alreadyReleased: false };
}
/**
```

- 「当前有客服权限且在线」的打手 user id —— **客服权限 + 在线判定的唯一出处**
- （csHall 的 onlineCsCount 与大厅广播共用，禁止第二份判定）。

*

- 口径与依据：
- - **有客服权限**：worker.status = 1 AND worker.can_serve_customer = 1（实时查库，无缓存；
- 与 workerOf / assertWorkerCs 同一张表同一判据）；
- - **在线**：沿用**既有机制** RealtimeGateway.isOnline(userId) —— 即 /ws/chat 命名空间里
- uid:{userId} 房间存在连接（好友列表的在线状态用的就是它）。**没有**另造心跳表、

```
===== 源文件：server/src/modules/chat/chat.service.ts （第 1015-1146 行）=====
----- 片段说明：消息发送：成员校验、限流、序号分配、实时推送 -----
async closeConversation(conversationId: string, actor: ChatActor) {
const conv = await this.convRepo.findOne({ where: { id: conversationId } });
if (!conv) throw new NotFoundException('会话不存在');
conv.status = 'closed';
conv.closedAt = new Date();
conv.closedBy = 'admin';
await this.convRepo.save(conv);
this.realtime.emitToUser(conv.userId, 'chat:closed', { conversationId });
}
// ---------- 消息（docs/07 核心） ----------
/** 发送限流：20 条/分/主体（Redis 计数，docs/07） */
private async checkRateLimit(actor: ChatActor): Promise<void> {
const key = `chat:rate:${actor.type}:${actor.userId}`;
const n = await this.redis.incr(key);
if (n === 1) await this.redis.expire(key, 60);
if (n > 20) {
throw new BadRequestException('发送过于频繁，请稍后再试');
}
}
/**
```

- 发送消息：
- - 事务内行锁会话 → seq = last_seq + 1 → 落库 → 更新 last_seq 与未读数（docs/07 顺序保证）
- - clientMsgId 幂等：唯一键冲突时回查原消息重发 ACK，不重复落库
- - 广播：msg:ack → 发送方；msg:new → 会话房间；unread:update → 对方

```
*/
async sendMessage(actor: ChatActor, conversationId: string, dto: { clientMsgId?: string; msgType?: string; content: string }) {
const content = (dto.content ?? '').trim().slice(0, 5000);
if (!content) throw new BadRequestException('消息内容为空');
const msgType = dto.msgType || 'text';
if (!['text', 'image'].includes(msgType)) throw new BadRequestException('不支持的消息类型');
await this.checkRateLimit(actor);
// 归属校验（docs/07 铁律）
const conv =
actor.type === 'user'
```

? await this.assertUserMember(conversationId, actor.userId)
: await this.assertWorkerCs(conversationId, actor.userId);
if (conv.status !== 'open') throw new BadRequestException('会话已关闭');
const clientMsgId = dto.clientMsgId ? String(dto.clientMsgId).slice(0, 64) : null;
// 幂等先行：同 clientMsgId 直接重发原 ACK
if (clientMsgId) {
const dup = await this.msgRepo.findOne({ where: { conversationId, clientMsgId } });
if (dup) {
this.realtime.emitToUser(actor.userId, 'msg:ack', this.ackView(dup));
return this.messageView(dup);
}
}
/**

- 「首次发消息进池」标记（负责人 2026-10-08）：客服会话里**老板的第一条消息**也要推 cs:new——
- 老板常常先打开客服页（会话已建）再打字，只靠"创建时广播"会让客服看不到内容变化。

*

- 取值在**事务内、行锁之后**读 locked.lastSeq：并发两条首消息被行锁串行化后，
- 只有真正的那一条看到 last_seq = 0 → 只广播一次（不是靠"猜"，是锁给的确定性）。

*/
let csFirstUserMessage = false;
const message = await this.convRepo.manager.transaction(async (em) => {
const locked = await em.findOne(ChatConversation, { where: { id: conversationId }, lock: { mode: 'pessimistic_write' } });
if (!locked || locked.status !== 'open') throw new BadRequestException('会话不可用');
const seq = Number(locked.lastSeq) + 1;
if (locked.type === 'cs' && actor.type === 'user' && Number(locked.lastSeq) === 0) csFirstUserMessage = true;
const msgId = randomUUID();
try {
const msg = await em.save(
em.create(ChatMessage, {
msgId, conversationId, seq: String(seq), senderType: actor.type, senderId: actor.userId,
msgType, content, clientMsgId,
}),
);
locked.lastSeq = String(seq);
// 未读归属：好友会话记对方侧；CS 会话用户发→客服侧，客服发→用户侧
if (actor.type === 'user' && locked.type === 'friend') {
if (String(locked.userId) === String(actor.userId)) locked.unreadPeer += 1;
else locked.unreadUser += 1;
} else if (actor.type === 'user') {
locked.unreadWorker += 1;
} else {
locked.unreadUser += 1;
}
await em.save(locked);
return msg;
} catch (e) {
// 并发同 clientMsgId：唯一键冲突 → 回查幂等重发（不重复落库）
const err = e as { driverError?: { code?: string } };
if (clientMsgId && err.driverError?.code === 'ER_DUP_ENTRY') {
const existing = await em.findOne(ChatMessage, { where: { conversationId, clientMsgId } });

```
if (existing) return existing;
}
throw e;
}
});
// 广播（docs/07）
this.realtime.emitToUser(actor.userId, 'msg:ack', this.ackView(message));
const senderNickname = (await this.nicknamesOf([actor.userId])).get(String(actor.userId)) ?? '';
this.realtime.server.to(`conv:${conversationId}`).emit('msg:new', this.messageView(message, senderNickname));
if (actor.type === 'user' && (conv.type === 'friend' || conv.type === 'worker')) {
// 好友/打手直聊：对方未读 +1 推送（事务后重读拿准确值）
const fresh = await this.convRepo.findOne({ where: { id: conversationId } });
const peerId = String(conv.userId) === String(actor.userId) ? conv.peerUserId : conv.userId;
if (fresh && peerId) {
const peerUnread = String(fresh.userId) === String(peerId) ? fresh.unreadUser : fresh.unreadPeer;
this.realtime.emitToUser(peerId, 'unread:update', { conversationId, unread: Number(peerUnread) });
}
} else if (actor.type === 'user') {
// CS：通知管理员 + 实时推给被指派客服（工作端列表立即刷新未读，修复调研缺口 A）
this.realtime.emitToAdmins('chat:activity', { conversationId });
const assigns = await this.assignRepo.find({ where: { conversationId, status: 'active' } });
for (const a of assigns) {
const fresh = await this.convRepo.findOne({ where: { id: conversationId } });
this.realtime.emitToUser(a.workerUserId, 'unread:update', { conversationId, unread: Number(fresh?.unreadWorker ?? 0) });
}
// 还没人接入 + 这是本会话的第一条消息 → 让在线客服的大厅立刻出现这条待接入（cs:new）
if (csFirstUserMessage && assigns.length === 0) {
void this.announceCsNew(conversationId, 'first-message', actor.userId);
}
} else {
this.realtime.emitToUser(conv.userId, 'unread:update', { conversationId, unread: conv.unreadUser + 1 });
}
return this.messageView(message);
}
/**
```

- 标记已读（docs/07 msg:read）：清自己侧未读 + 记录**自己的已读位点**，
- 并把「对方读到哪一条」推给会话内另一方（负责人 2026-10-04：单勾=已送达 / 双勾=已读）。

*

- 位点取服务端数据：该会话内「非系统消息、非本人发送」的最大 seq —— 即读者确实收到的对方消息里最新的一条；

```
===== 源文件：server/src/modules/recharge/recharge.service.ts （第 369-920 行）=====
----- 片段说明：充值人工审核：待审列表、发放（复用积分发放事务）、驳回、收款码管理 -----
async reviewList(query: ReviewQuery, page: number, pageSize: number) {
const qb = this.dataSource
.createQueryBuilder(RechargeRequest, 'r')
.innerJoin(User, 'u', 'u.id = r.user_id');
const status = String(query.status ?? '').trim();
if (status && status !== 'all') {
if (!['pending', 'granted', 'rejected'].includes(status)) {
throw new BadRequestException('状态筛选值不合法（pending/granted/rejected/all）');
}
qb.andWhere('r.status = :status', { status });
}
--- 第 50 页 ---
```

```
const keyword = String(query.keyword ?? '').trim();
if (keyword) {
qb.andWhere('(u.username LIKE :kw OR u.nickname LIKE :kw OR u.uid LIKE :kw)', { kw: `%${keyword}%` });
}
const start = this.parseTime(query.startTime, '开始时间');
if (start) qb.andWhere('r.created_at >= :start', { start });
const end = this.parseTime(query.endTime, '结束时间');
if (end) qb.andWhere('r.created_at <= :end', { end });
const total = Number((await qb.clone().select('COUNT(1)', 'cnt').getRawOne())?.cnt ?? 0);
const list = await qb
.orderBy('r.id', 'DESC')
.offset((page - 1) * pageSize)
.limit(pageSize)
.getMany();
return {
list: await this.decorate(list),
total,
page,
pageSize,
pendingTotal: await this.requestRepo.count({ where: { status: PENDING_STATUS } }),
};
}
/** 受限会话身份卡片：GET /recharge/me */
async adminMe(userId: string) {
const user = await this.userRepo.findOne({ where: { id: String(userId) } });
if (!user) throw new NotFoundException('账号不存在');
return {
admin: {
id: String(user.id),
nickname: user.nickname || user.username,
username: user.username,
},
kind: 'recharge' as const,
pendingTotal: await this.requestRepo.count({ where: { status: PENDING_STATUS } }),
};
}
/**
```

- `POST /recharge/requests/:id/grant` —— **幂等发放**（本功能最关键的一段）。

*

- 事务内顺序（每一步都有理由，改这里之前先读 CLAUDE.md 结构规则 12）：
- 1. `SELECT ... FOR UPDATE` 锁住这张申请 → 并发/重复点击在此串行化；
- 2. `status !== 'pending'` → 中文 400（已发放/已驳回都不允许再动）；
- 3. `WalletService.grantInTx(em, ...)` —— **复用既有超管发积分能力**，同一个事务：
- 行锁钱包 → 改余额 → 写流水（含 `balance_before/after` 快照）；
- 幂等键 `recharge:<requestId>` 由 `wallet_transaction.idempotency_key` 唯一约束兜底；
- 4. 回读该流水（拿 id 与前后余额）→ 是**同一事务**里读的，看到的一定是自己这一步写的值；
- 5. **条件 UPDATE** 回写 `status='granted' / reviewed_by / reviewed_at / wallet_transaction_id`，
- 判 `affectedRows === 1`；不为 1（理论上不可能：行锁还在手上）→ 抛错整体回滚，**绝不留半成品**；
- 6. 通知用户（站内通知与授予同事务，提交后推实时事件）。

*

- 重复调用会发生什么（负责人要求的证据点）：第 2 步命中 granted → 中文 400「该申请已发放」，
- **不进第 3 步，积分一分不加。**

```
*/
async grant(requestId: string, adminUserId: string, meta: ReviewMeta) {
const outcome = await this.dataSource.transaction(async (em: EntityManager) => {
const row = await this.lockRequest(em, requestId);
this.assertPending(row.status);
const idempotencyKey = recharge:${row.id};
// ③ 复用既有发积分能力（同一事务；不新写一套积分逻辑）
await this.wallet.grantInTx(
em,
String(adminUserId),
{
userId: String(row.userId),
amount: Number(row.points),
reason: 充核放：申 #${row.id}（${Number(row.amountYuan)} 元），
idempotencyKey,
},
meta,
);
// ④ 回读流水（同一事务内，必然是本步刚写的那条）
const txn = await this.walletTransactionOf(em, idempotencyKey);
if (!txn) {
throw new BadRequestException('积分流水写入失败，请重试');
}
// ⑤ 条件 UPDATE 判 affectedRows
const res: { affectedRows?: number } = await em.query(
UPDATE \recharge_request\`
SET \status\ = 'granted', \reviewed_by\ = ?, \reviewed_at\ = NOW(3),
\wallet_transaction_id\ = ?, \reject_reason\ = NULL
WHERE \id\ = ? AND \status\ = 'pending'`,
[String(adminUserId), String(txn.id), String(row.id)],
);
if (Number(res?.affectedRows ?? 0) !== 1) {
throw new BadRequestException('该申请已发放');
}
// ⑥ 站内通知（与授予同事务：授予成功就一定有一条"已到账"通知，不会漏）
await em.save(
em.create(UserNotification, {
userId: String(row.userId),
type: 'recharge_granted',
title: '充值申请已到账',
content: 充值申请已到账：+${Number(row.points)} 积分,
refId: String(row.id),
}),
);
return {
requestId: String(row.id),
userId: String(row.userId),
points: Number(row.points),
```

```
amountYuan: Number(row.amountYuan),
transactionNo: txn.transactionNo,
walletTransactionId: String(txn.id),
balanceBefore: Number(txn.balanceBefore),
balanceAfter: Number(txn.balanceAfter),
};
});
// 事务提交后再推实时事件（回滚时不该推送）
this.realtime.emitToUser(outcome.userId, 'notification:new', { type: 'recharge_granted' });
this.realtime.emitToUser(outcome.userId, 'recharge:granted', {
requestId: outcome.requestId,
points: outcome.points,
});
this.logger.log(
充值发放 requestId=${outcome.requestId} user=${outcome.userId} +${outcome.points} 积分 +
流水=${outcome.transactionNo} 余额 ${outcome.balanceBefore}→${outcome.balanceAfter} by=${adminUserId},
);
return {
ok: true,
id: outcome.requestId,
status: 'granted' as const,
points: outcome.points,
transactionNo: outcome.transactionNo,
walletTransactionId: outcome.walletTransactionId,
balanceBefore: outcome.balanceBefore,
balanceAfter: outcome.balanceAfter,
};
}
/**
```

- `POST /recharge/requests/:id/reject` —— 驳回（同样事务 + 行锁 + 条件 UPDATE，防重复处理）。
- 驳回**不触碰积分**（没有可回滚的东西：pending 申请从未加过分）。

```
*/
async reject(requestId: string, adminUserId: string, reason: string) {
const trimmed = String(reason ?? '').trim();
if (!trimmed) throw new BadRequestException('请填写驳回原因');
if (trimmed.length > 200) throw new BadRequestException('驳回原因最多 200 字');
const outcome = await this.dataSource.transaction(async (em: EntityManager) => {
const row = await this.lockRequest(em, requestId);
this.assertPending(row.status);
const res: { affectedRows?: number } = await em.query(
UPDATE \recharge_request\`
SET \status\ = 'rejected', \reviewed_by\ = ?, \reviewed_at\ = NOW(3), \reject_reason\ = ?
WHERE \id\ = ? AND \status\ = 'pending'`,
[String(adminUserId), trimmed, String(row.id)],
);
if (Number(res?.affectedRows ?? 0) !== 1) {
throw new BadRequestException('该申请已处理，不能重复操作');
}
await em.save(
em.create(UserNotification, {
```

```
userId: String(row.userId),
type: 'recharge_rejected',
title: '充值申请被驳回',
content: 你的充值申请（${Number(row.amountYuan)} 元）被驳回：${trimmed},
refId: String(row.id),
}),
);
return { requestId: String(row.id), userId: String(row.userId), amountYuan: Number(row.amountYuan) };
});
this.realtime.emitToUser(outcome.userId, 'notification:new', { type: 'recharge_rejected' });
this.logger.log(充值驳回 requestId=${outcome.requestId} user=${outcome.userId} by=${adminUserId} 原因=${trimmed});
return { ok: true, id: outcome.requestId, status: 'rejected' as const, rejectReason: trimmed };
}
/**
```

- `GET /recharge/proof/:requestId` —— 受限会话读取"这张申请的那张转账截图"。

*

- 为什么不复用 `/media/protected/:id`：那个接口对 `role='admin'` 放行**任意**受保护媒体
- （聊天图片也在内），而受限会话的白名单里刻意没有 `/media/*`。
- 这里把权限面收窄到"这条充值申请对应的这一张图"，且必须持有 `recharge:review`（Controller 层校验）。

```
*/
async proofMedia(requestId: string): Promise<Media> {
const row = await this.requestRepo.findOne({ where: { id: String(requestId) } });
if (!row) throw new NotFoundException('充值申请不存在');
const media = await this.mediaRepo.findOne({ where: { id: String(row.proofMediaId) } });
if (!media) throw new NotFoundException('转账截图不存在');
return media;
}
// ============================================================================
// 收款码（管理端）
// ============================================================================
/** GET /admin/recharge/qr-codes：enabled 不传 = 全部（管理端要看停用的） */
async listQrCodes(enabled?: string) {
const where = enabled === '1' || enabled === '0' ? { enabled: Number(enabled) } : {};
const list = await this.qrRepo.find({ where, order: { sort: 'ASC', id: 'ASC' } });
const paths = await this.media.pathsByIds(list.map((q) => String(q.mediaId)));
return {
list: list.map((q) => ({
id: String(q.id),
mediaId: String(q.mediaId),
url: paths.get(String(q.mediaId)) ?? null,
channel: q.channel,
label: q.label,
sort: q.sort,
enabled: q.enabled === 1,
createdAt: q.createdAt,
updatedAt: q.updatedAt,
})),
total: list.length,
};
}
```

```
/**
```

- POST /admin/recharge/qr-codes：新增收款码（图片先走既有媒体上传拿 mediaId）。

```
*
```

- 入参兼容 imageMediaId（管理端前端 api/recharge.ts 的契约字段名）与 mediaId（后端列名）
- —— 两个名字指同一个东西，收敛在一处，避免"前后端各叫一个名字"变成线上 400。

```
*/
async createQrCode(dto: {
mediaId?: string;
imageMediaId?: string;
channel?: string;
label?: string;
sort?: number;
enabled?: boolean;
}) {
const mediaId = String(dto.imageMediaId ?? dto.mediaId ?? '').trim();
await this.assertMediaImage(mediaId, '收款码图片');
const row = await this.qrRepo.save(
this.qrRepo.create({
mediaId,
channel: this.normalizeChannel(dto.channel),
label: (dto.label ?? '').slice(0, 64),
sort: Number.isFinite(Number(dto.sort)) ? Number(dto.sort) : 0,
enabled: dto.enabled === false ? 0 : 1,
}),
);
this.invalidatePublicConfigCache();
this.logger.log(收款已新增 id=${row.id} channel=${row.channel} media=${row.mediaId});
return {
id: String(row.id),
mediaId: String(row.mediaId),
imageMediaId: String(row.mediaId),
channel: row.channel,
label: row.label,
sort: row.sort,
enabled: row.enabled === 1,
};
}
/** PUT /admin/recharge/qr-codes/:id：改图/渠道/名称/排序/启停（部分提交；接受两种图片字段名） */
async updateQrCode(
id: string,
dto: {
mediaId?: string;
imageMediaId?: string;
channel?: string;
label?: string;
sort?: number;
enabled?: boolean;
},
) {
const row = await this.qrRepo.findOne({ where: { id: String(id) } });
```

```
if (!row) throw new NotFoundException('收款码不存在');
const nextMediaId = dto.imageMediaId ?? dto.mediaId;
if (nextMediaId !== undefined) {
await this.assertMediaImage(nextMediaId, '收款码图片');
row.mediaId = String(nextMediaId);
}
if (dto.channel !== undefined) row.channel = this.normalizeChannel(dto.channel);
if (dto.label !== undefined) row.label = String(dto.label).slice(0, 64);
if (dto.sort !== undefined && Number.isFinite(Number(dto.sort))) row.sort = Number(dto.sort);
if (dto.enabled !== undefined) row.enabled = dto.enabled ? 1 : 0;
const saved = await this.qrRepo.save(row);
this.invalidatePublicConfigCache();
return {
id: String(saved.id),
mediaId: String(saved.mediaId),
imageMediaId: String(saved.mediaId),
channel: saved.channel,
label: saved.label,
enabled: saved.enabled === 1,
sort: saved.sort,
};
}
/**
```

- `DELETE /admin/recharge/qr-codes/:id`：**物理删除。**

```
*
```

- 与"停用不删"的口径区分：停用（`enabled=0`）用于"暂时不收这张码"，行保留；
- 删除用于"传错了/作废了"。历史申请引用的是 `qr_id`（可空），删码不会破坏历史 ——
- 列表侧对已删的码回落为 `qrLabel=null`，不会 500。

```
*/
async deleteQrCode(id: string) {
const row = await this.qrRepo.findOne({ where: { id: String(id) } });
if (!row) throw new NotFoundException('收款码不存在');
await this.qrRepo.remove(row);
this.invalidatePublicConfigCache();
this.logger.log(收款已除 id=${id} media=${row.mediaId});
return { deleted: true, id: String(id) };
}
// ============================================================================
// 受限管理员白名单（管理端；仅超管可增删）
// ============================================================================
/** GET /admin/recharge/admins：白名单 + 上限 + 可选账号（角色必须是 admin） */
async listAdmins() {
const cfg = await this.config();
const rows: Array<{
userId: string | number;
username: string;
nickname: string;
uid: string;
status: number;
grantedBy: string | number | null;
```

```
createdAt: Date | null;
roleCode: string | null;
isOwner: number | null;
}> = await this.dataSource.query(
`SELECT u.id AS userId, u.username, u.nickname, u.uid, u.status,
ra.granted_by AS grantedBy, ra.created_at AS createdAt,
ar.code AS roleCode, a.is_owner AS isOwner
FROM recharge_admin ra
JOIN user u ON u.id = ra.user_id
LEFT JOIN admin a ON a.user_id = u.id
LEFT JOIN admin_role ar ON ar.id = a.role_id
ORDER BY ra.created_at ASC, u.id ASC`,
);
return {
list: rows.map((r) => ({
/** 前端删条目用这个键；recharge_admin 没有代理主键，故 id === adminUserId */
id: String(r.userId),
adminUserId: String(r.userId),
userId: String(r.userId),
username: r.username,
nickname: r.nickname,
uid: r.uid,
status: Number(r.status),
roleCode: r.roleCode ?? null,
isOwner: Number(r.isOwner ?? 0) === 1,
grantedBy: r.grantedBy === null ? null : String(r.grantedBy),
createdAt: r.createdAt,
})),
total: rows.length,
/** 当前生效上限（= min(配置, 硬上限 5)） */
adminLimit: cfg.adminLimit,
/** 硬上限（负责人定案：最多 5 个账号；配置只能在本值以内收紧） */
adminLimitMax: RECHARGE_ADMIN_LIMIT_MAX,
};
}
/**
```

- `POST /admin/recharge/admins` —— 加入白名单（**仅超管**，调用方已校验）。

*

- 入参兼容 `adminUserId`（管理端前端契约字段名）/ `targetUserId` / `userId`。

*

- 并发安全（负责人明确要求"唯一约束 + 事务 + 行锁"）：
- - **事务**：计数与插入必须在同一临界区；
- - **行锁**：`SELECT setting_key FROM system_setting WHERE setting_key='recharge_config' FOR UPDATE`
- —— 所有"加入/移出白名单"的操作在这**同一行**上排队，因此"读到的条数"与"插入"之间
- 不可能插入另一个并发加入。为什么锁配置行而不是锁 user 行：要防的是"两个**不同**账号
- 同时加入把总数顶过上限"，锁目标账号各自的行起不到串行化作用；
- - **唯一约束**：`recharge_admin.user_id` 是主键 → 同一账号不可能加两次；
- - 撞唯一键（并发加同一账号）→ 中文 400，而不是 500。

```
*/
async addAdmin(callerUserId: string, targetUserId: string) {
```

软件名称：祁连山电竞陪玩服务平台软件 V1.0
```
const targetId = String(targetUserId ?? '').trim();
if (!targetId) throw new BadRequestException('请选择要授权的管理员账号');
const result = await this.dataSource.transaction(async (em: EntityManager) => {
// 配置行行锁：白名单的增删全部在这一行上排队（并发安全的关键）
await em.query('SELECT `setting_key` FROM `system_setting` WHERE `setting_key` = ? FOR UPDATE', [
RECHARGE_CONFIG_KEY,
]);
const target = await em.findOne(User, { where: { id: targetId } });
if (!target) throw new NotFoundException('账号不存在');
// 只允许 role=admin 的账号（负责人定案：白名单里只能是管理员账号）
if (target.role !== 'admin') throw new BadRequestException('该账号不是管理员账号，无法授权充值审核');
const profile = await em.query(
`SELECT a.status, ar.code AS roleCode FROM admin a LEFT JOIN admin_role ar ON ar.id = a.role_id
WHERE a.user_id = ? LIMIT 1`,
[targetId],
);
if (!profile.length) throw new BadRequestException('该账号没有管理员档案，无法授权充值审核');
if (Number(profile[0].status) !== 1) throw new BadRequestException('该管理员账号已停用，请先启用');
const existing = await em.findOne(RechargeAdmin, { where: { userId: targetId } });
if (existing) throw new BadRequestException('该账号已在充值审核白名单中');
// 上限：配置值（读侧已按硬上限 5 收敛）+ 再兜一次硬上限（不依赖配置正确性）
const cfg = await this.config();
const limit = Math.min(cfg.adminLimit, RECHARGE_ADMIN_LIMIT_MAX);
const countRows: Array<{ c: number | string }> = await em.query(
'SELECT COUNT(1) AS c FROM recharge_admin',
);
const current = Number(countRows[0]?.c ?? 0);
if (current >= limit) {
throw new BadRequestException(
`充值审核管理员已达上限（${limit} 个，最多 ${RECHARGE_ADMIN_LIMIT_MAX} 个），请先移除后再添加`,
);
}
try {
await em.save(em.create(RechargeAdmin, { userId: targetId, grantedBy: String(callerUserId) }));
} catch (e) {
// 并发撞主键：翻译成中文 400（而不是 500）
const code = (e as { code?: string; driverError?: { code?: string } })?.code
?? (e as { driverError?: { code?: string } })?.driverError?.code;
if (code === 'ER_DUP_ENTRY') throw new BadRequestException('该账号已在充值审核白名单中');
throw e;
}
return { userId: targetId, username: target.username, count: current + 1, limit };
});
this.logger.log(`充值审核白名单 +1 user=${result.userId} by=${callerUserId}（${result.count}/${result.limit}）`);
return {
ok: true,
userId: result.userId,
username: result.username,
total: result.count,
adminLimit: result.limit,
```

};
}
/**

- DELETE /admin/recharge/admins/:userId —— 移出白名单（**仅超管**，调用方已校验）。

*

- **移出即失效**（负责人要求的验证点 5）：删白名单行 + **立即吊销该账号全部 `kind='recharge'` 会话**
- + 断开其 WS（若有）。下一次请求 checkRechargeSession 查不到白名单行 → **401 中文。**
- **绝不动 primary**：管理员在管理端的登录照常可用（AuthService.revokeRechargeTokens 只命中 recharge）。

*/
async removeAdmin(callerUserId: string, targetUserId: string) {
const targetId = String(targetUserId ?? '').trim();
if (!targetId) throw new BadRequestException('请选择要移除的管理员账号');
const row = await this.adminRepo.findOne({ where: { userId: targetId } });
if (!row) throw new NotFoundException('该账号不在充值审核白名单中');
await this.adminRepo.remove(row);
// 立即吊销受限会话（只吊销 recharge）+ 主动断开其 WS（当前握手阶段就拒绝 recharge，
// 因此正常情况下踢到 0 个；保留这一步是为了"将来若有人放开 WS，踢线不会漏"）
const revoked = await this.auth.revokeRechargeTokens(targetId);
const kicked = this.realtime.disconnectByJti(revoked.jtis, 'auth:expired');
// 通知被移除的管理员（站内通知；不改动其 primary 任何东西）
await this.notify(targetId, {
type: 'recharge_admin_removed',
title: '充值审核权限已收回',
content: '你的充值审核（受限管理）权限已被超级管理员收回，如需继续请重新申请。',
refId: null,
});
const countRows: Array<{ c: number | string }> = await this.dataSource.query('SELECT COUNT(1) AS c FROM recharge_admin');
this.logger.log(
`充值审核白名单 -1 user=${targetId} by=${callerUserId} 吊销受限会话=${revoked.affected} 断开 WS=${kicked}`,
);
return {
ok: true,
userId: targetId,
total: Number(countRows[0]?.c ?? 0),
revokedSessions: revoked.affected,
message: '已移出白名单，该账号的受限管理身份立即失效',
};
}
// ==
// 绑定受限管理员态（老板端「身份」）
// ==
/**

- POST /user/identity/bind-recharge-admin —— 用**管理员账密**换取受限管理会话。

*

- 与 /user/identity/bind（打手态）逐条同构，差别只在"校验对象是 admin 白名单"：
- 1. 限流（5 次/分/IP+账号，防管理员账密爆破）→ 429 中文；
- 2. 定位账号 → 查不到：**与密码错逐字同一句**「账号或密码错误」（防账号枚举，安全审计 F2 口径）；
- 3. bcrypt 校验（AuthService.comparePassword，与登录同一实现）→ 错：同一句；
- 4. 角色必须是 admin → 否则 403「该账号不是管理员账号，无法进入管理员态」；
- 5. 封禁（复用 AuthService.assertNotBanned：到期自动解封 / 封禁中 403 + ACCOUNT_BANNED）；

- 6. **必须在 `recharge_admin` 白名单内** → 否则 403 中文（**先验密码再看白名单**：
- 不知道密码的人得不到任何"某账号是否被授权"的情报）；
- 7. 签发 `kind='recharge'` 受限会话（**不顶 primary**；TTL 12h，与 companion 一致）。

*

- 主体 id 只取 JWT sub（调用方是谁不影响签发结果——签发对象永远是"被验证账密的那个管理员账号"）。

```
*/
async bindRechargeAdmin(
callerUserId: string,
dto: { username?: string; password?: string },
ip: string,
userAgent: string,
) {
const account = String(dto?.username ?? '').trim();
if (!account) throw new BadRequestException('请填写管理员账号');
if (!dto?.password) throw new BadRequestException('请填写密码');
if (!(await this.rateLimit.allow(`recharge-bind:${ip}:${account}`, RECHARGE_BIND_RATE_LIMIT, RECHARGE_BIND_RATE_WINDOW_SECONDS))) {
await this.auth.recordIdentityEvent(null, account, ip, userAgent, 'fail', '受限管理绑定：触发限流');
throw new HttpException(
{ message: '尝试过于频繁，请稍后再试', errorCode: 'RATE_LIMITED' },
HttpStatus.TOO_MANY_REQUESTS,
);
}
const target = await this.userRepo
.createQueryBuilder('u')
.addSelect('u.passwordHash')
.where('u.username = :account OR u.uid = :account', { account })
.getOne();
if (!target) {
await this.auth.recordIdentityEvent(null, account, ip, userAgent, 'fail', '受限管理绑定：账号不存在');
throw new BadRequestException('账号或密码错误');
}
if (!(await this.auth.comparePassword(target, dto.password))) {
await this.auth.recordIdentityEvent(target.id, target.username, ip, userAgent, 'fail', '受限管理绑定：密码错误');
throw new BadRequestException('账号或密码错误');
}
if (target.role !== 'admin') {
await this.auth.recordIdentityEvent(target.id, target.username, ip, userAgent, 'fail', '受限管理绑定：角色不是管理员');
throw new ForbiddenException('该账号不是管理员账号，无法进入管理员态');
}
await this.auth.assertNotBanned(target, 'identity', target.username, ip, userAgent);
const whitelisted = await this.adminRepo.findOne({ where: { userId: String(target.id) } });
if (!whitelisted) {
await this.auth.recordIdentityEvent(target.id, target.username, ip, userAgent, 'fail', '受限管理绑定：不在白名单');
throw new ForbiddenException('该账号未获授权进行充值审核，请联系超级管理员');
}
const tokens = await this.auth.issueRechargeTokens(target);
await this.auth.recordIdentityEvent(
target.id,
target.username,
ip,
```

源码清洗说明（送审前已处理项）

为保证送审材料不含敏感信息，本文档所展示的源码在**不改变任何业务语句**的前提下做了以下处理：

序号	清洗项	处理方式	核查结果（在 server/src/** 全量检索）
1	密钥 / 令牌 / 口令明文	源码中不存在任何硬编码密钥；用户口令一律经 bcrypt 哈希后入库，JWT / 数据库 / 邮件密钥一律由环境变量注入（代码里只出现变量名，不出现取值）	检索 secret / password / apiKey 赋值字面量 → **0 命中**
2	内网地址与真实域名	源码中不写死任何隧道域名或公网域名	检索内网网段（192.168 / 10.0 / 172.16-31）与隧道域名 → **0 命中**
3	本机回环地址	清洗规则已启用（127.0.0.1 → <本机回环地址>）；server/src/** 全文共 7 处，**本次选入正文的片段中经逐行核查为 0 处**，因此正文中只出现替换后的标记	替换后检索 127.0.0.1 → **0 命中**
4	环境变量访问语句	process.env.XXX 属程序逻辑（只出现变量名、不含取值），**刻意保留不改**，避免破坏语句合法性	保留 15 处，全部为变量名
5	真实账号 / 手机号 / 证件号	源码中不含任何真实账号、手机号、身份证号	检索手机号形态 → **0 命中**
6	第三方 App 素材与文案	源码与注释中不含任何第三方 App 的图片、插画、吉祥物或文案	项目规范明令禁止复制 → 检索 **0 命中**
7	业务数据	本文档展示的是程序逻辑源码，不含数据库中的真实用户数据（昵称、工号等测试期拟真数据不出现在源码中）	已确认

复现方式：以上检索均可在仓库根目录用同样的关键字重跑（Select-String -Path "server\src" -Include *.ts -Pattern "<关键字>"）。

页数与行数校验

校验项	要求	实际	结论
前部页数	30	30	符合

后部页数	30	30	符合
每页非空行数	50	逐页 50（脚本断言，不足或超出即中止生成）	符合
总页数	60	60	符合
总展示行数	3000	3000	符合

行数口径说明：本文件按「非空行」计数（每页 50 个非空行），与「源程序量按有效代码行统计」的常见口径一致。若受理机构要求按含空行的物理行排版，可改用仓库原始文件直接排版（同一批文件按物理行统计见《软著申请表信息.md》）。